

Software Requirements Specification

For

Dishari

Prepared by

Md Sabbir Ahamed ASH2225005M

Fatama Beg Tuhin BFH2225016F

Mohammad Mahmudul Hasan MUH2225029M

Nelema Jahan BFH2125032F

**Institute of Information and Technology
Noakhali Science and Technology University**

21.09.2025

Table of Contents

1. Introduction	1
1.1 Problem Statement	1
1.2 Purpose	1
1.3 Project Scope	2
1.4 Glossary	2
1.5 References	3
1.6 Overview	3
2. Overall Description	3
2.1 Product Perspective	3
2.2 Productive Functions	4
2.2.1 Patient Module:	4
2.2.2 Doctor Module:	4
2.2.3 Dispenser Module:	4
2.2.4 Lab Module:	5
2.2.5 Administrative Module:	5
3. Stake Holders and Characteristics	5
3.1 Primary Stakeholders	5
3.1.1 Project Owner / University Authority	5
3.1.2 Development Team	6
3.1.3 UI/UX Designers	6
3.1.4 QA Testers	6
3.1.5 System Administrator / DevOps Team	7
3.2 Secondary Stakeholders	7
3.2.1 Patients (Students/Faculty/Others)	7
3.2.2 Doctors	7
3.2.3 Dispensers	8
3.2.4 Lab Staff	8
3.2.5 Administrators (End-user Admins)	8
3.3 Stakeholder Matrix:	9
4 Design and Implementation Constraints	9
4.1 System Architecture	9

4.1.1 Client-side (Mobile App)	9
4.1.2 Server-side (Backend).....	9
4.2 Development Languages and Tools	10
4.2.1 Framework	10
4.2.2 Database System	10
4.2.3 Tools	10
4.2.4 Platform and Deployment Constraints	10
4.2.5 Security & Compliance Constraints.....	11
4.2.6 Design Constraints	11
5 User Documentation	11
5.1 Introduction.....	11
5.1.1 User Roles and Access.....	11
5.1.2 Patient Use Guide.....	12
5.1.3 Doctor Use Guide.....	12
5.1.4 Medicine Dispenser Use Guide.....	13
5.1.5 Lab Staff User Guide	13
5.1.6 Admin Use Guide.....	14
6. Functional Requirements	15
6.1. User Registration and Login	15
6.2. Profile Management.....	15
6.3. Prescription Management	15
6.4. Medicine Dispensing	15
6.5. Lab Test Management.....	16
6.6. External Test Upload	16
6.7. Notifications.....	16
6.8. Ratings	17
6.9. Admin controls.....	17
6.10. Staff Rostering	17
6.11. Payments.....	17
6.12. Audit & History	17
7.Non-Functional Requirements.....	18
7.1. Performance	18
7.2. Scalability	18
7.3. Security	18

7.4. Availability & Reliability	19
7.5. Usability.....	19
7.6. Maintainability & Extensibility	19
7.7. Auditability & Compliance.....	19
7.8. Portability.....	20
8.Data Requirements	20
8.1. User Data	20
8.2. Patient Medical Profile Data.....	20
8.3 Prescription Data.....	21
8.4 Medicine Inventory Data	21
8.5 Lab Test Data.....	21
8.6 Doctor Data.....	21
8.7 Notification Data.....	22
8.8 Feedback & Rating Data.....	22
8.9 Admin & System Management Data	22
8.10 System Logs.....	22
9. Requirement Prioritization	22
9.1 Prioritization Table	23
9.2 Requirement Engineering Process	23
9.3 Requirement Elicitation Techniques.....	24
9.4 Hold Interviews.....	24
9.5 Sample of Requirement Collection	24
9.5.1. Requirement Collection – Students (Patients)	24
9.5.2. Requirement Collection – Doctors.....	25
9.5.3. Requirement Collection – Dispensers.....	25
9.5.4. Requirement Collection – Lab Staff	26
9.6 Requirement Validation	26
9.6.1 Walkthrough	26
9.6.2 Requirement Review.....	27
9.6.3. User Acceptance Testing (UAT).....	27
9.6.4 Requirements Traceability	27
10.UML Diagrams for Dishari.....	29
10.1 Class Diagram for Dishari	29
10.2 Use Case Diagram.....	30

10.3 Use Case description.....	31
10.3.1. Patient	31
10.3.2 Doctor	35
10.3.3 Lab Test	38
10.3.4 Dispenser.....	40
10.3.5 Admin	41
10.4 Activity Diagram	45
10.5 Sequence Diagram	52
10.6 Database.....	59

List of Figure

Figure 1: Class Diagram	29
Figure 2: Use Case Diagram	30
Figure 3: Patient registration.....	45
Figure 4: Patient login.....	45
Figure 5: Patient profile update.....	46
Figure 6: Patient prescriptions (view or download).....	46
Figure 7: Patient Upload External Test Results	47
Figure 8: Patient Check Lab Test Availability.....	47
Figure 9: Rate Doctor Same as Others	48
Figure 10: See Staff Information	48
Figure 11: Doctor Log In (Lab and Dispenser login same as Doctor)	49
Figure 12: Create/Edit/Print Prescriptions using Templates	49
Figure 13: Check Medicine Stock, Suggest Alternatives and dispenser.....	50
Figure 14: Admin Activity diagram.....	51
Figure 15: Patient Registration	52
Figure 16: Patient Login	53
Figure 17: Doctor Creating Prescription.....	54
Figure 18: Medicine Dispensing	55
Figure 19: Lab Test Processing.....	56
Figure 20: Emergency Case Handling	57
Figure 21: Patient Rating System.....	58
Figure 22: Admin Managing Inventory	58
Figure 23: Database Schema Diagram.....	59

1. Introduction

Dishari (It will show a patient's journey from illness to recovery, in mental or physical health.) is a comprehensive digital healthcare platform developed to enhance the medical services provided within Noakhali Science and Technology University (NSTU). The system integrates patient record management, prescription handling, medicine dispensing, laboratory test management, and administrative oversight into a single, unified solution. By digitizing and automating the entire healthcare workflow, the platform reduces errors, improves efficiency, ensures transparency, and fosters a patient-centered medical ecosystem.

1.1 Problem Statement

University healthcare services often believe on manual record-keeping, fragmented communication, and traditional processes that are inefficient and error-prone. Patients, doctors, dispensers, and administrators face challenges due to the lack of a unified, digital system that integrates all aspects of healthcare management. This results in miscommunication, lost records, limited accessibility, and reduced quality of care for both students and staff. The absence of real-time reporting also restricts the ability of healthcare administrators to plan effectively.

Specific Issues:

- **Fragmented Patient Records:** Medical records are scattered or paper-based, making it difficult for patients and doctors to access accurate, up-to-date information.
- **Prescription Inefficiencies:** Patients often lose paper prescriptions, while doctors and dispensers face challenges in tracking medicine availability and updates.
- **Medicine Stock Management:** Lack of digital inventory tracking leads to stock-outs, delays, and errors in medicine dispensing.
- **Weak Communication:** Absence of real-time notifications and reminders results in missed updates on prescriptions.
- **Lack of Analytics:** Administrators lack tools to generate reports on medicine usage and prescription patterns, limiting data-driven decision-making.
- **Minimal Transparency:** Patients have limited visibility of staff information and no formal system to provide feedback.

1.2 Purpose

The purpose of Dishari is to provide a unified digital healthcare platform for Noakhali Science and Technology University (NSTU) that streamlines patient care, medical record management, and administrative oversight. The system aims to reduce errors, save time, and ensure transparency by digitizing every aspect of healthcare operation, including patient registration, prescriptions, medicine dispensing, laboratory management, and reporting.

In essence, the purpose of Dishari is to create a secure, scalable, and patient-centered healthcare ecosystem within NSTU, fostering efficiency, reliability, and better health outcomes for students, faculty, and staff.

1.3 Project Scope

We are going to create a comprehensive platform for Noakhali Science and Technology University (NSTU) to improve the efficiency and quality of campus healthcare services. The system will be designed, implemented, and deployed as a unified digital solution that integrates patient management, prescriptions, medicine dispensing, laboratory services, and administrative oversight.

This project will cover the following areas: -

- **Secure login and registration:** for all users with role-based access.
- **Patient profiles:** full-name, store contact info, department, session, blood group, allergies, and medical history.
- **Prescriptions:** doctors can create, update, view, print, and send prescriptions to patients.
- **Lab tests:** request, submit, process, and view results; includes payment and notifications.
- **Medicine dispensing:** check stock, suggest alternatives, dispense medicines, and update inventory automatically.
- **Notifications:** reminders for new prescriptions, lab results, and test availability.
- **Admin features:** manage users, inventory, reports, and staff rostering.
- **Emergency cases:** doctors can handle urgent patients outside the university.
- **Directory:** view all staff and university ambulance contact.
- **Data security:** all information is kept safe and encrypted.
- **User-friendly interface:** simple design for mobile (android and iOS) use.

1.4 Glossary

This section provides definitions for all document names, acronyms, and abbreviations used in this SRS. Terms and concepts specific to the application domain are defined for further clarity.

Term/Acronym	Description/definition
SRS	Software Requirements Specification.
UI/UX	User Interface / User Experience.
DB	Database.
IDE	Integrated Development Environment (e.g., online text/code editor).
Flutter	An open-source UI toolkit by Google to build natively compiled mobile apps.
Serverpod	A server-side framework for Dart/Flutter to create backend APIs and manage databases.
MySQL	An open-source relational database management system

GitHub	A code hosting platform for version control and collaboration.
API	Application Programming Interface – a set of functions to communicate between client and server.

1.5 References

- IEEE. IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Computer Society, 1998.
- Software Engineering 9th Edition by Lan Sommerville.
- Requirements Engineering Fundamentals by Klaus Pohl.
- Database System Concepts 6th Edition by Abraham Silber Schatz

1.6 Overview

Our aim with the Dishari project is to create a unified digital healthcare platform for NSTU that will modernize and streamline campus medical services. We want to simplify patient management, prescription handling, laboratory services, and medicine dispensing, while making healthcare more secure and accessible for both registered and guest patients. Through Dishari, we aim to provide doctors and medical staff with efficient tools for creating prescriptions, managing schedules, and accessing reports. At the same time, administrators will be able to manage staff, inventory, and generate detailed reports more effectively. Ultimately, our goal is to reduce errors, save time, ensure transparency, and establish a reliable, patient-centered healthcare ecosystem within the university.

2. Overall Description

2.1 Product Perspective

Dishari is a cross-platform healthcare management system consisting of a mobile application for patients and a web-based portal for doctors, dispensers, lab staff, and administrators. It follows a **modular client-server architecture**, where the mobile frontend communicates with the backend server via **Serverpod**.

While the system is self-contained, it may integrate with:

- **Email gateways** for account verification and notifications.
- **University payment gateways** for lab test payments.
- **External notification services** (e.g., SMS).

The product will be developed using the following technology stack:

- **Frontend (Mobile App):** Flutter – for building cross-platform mobile applications with native performance.
- **Backend:** Serverpod – for managing APIs, database operations, authentication, and business logic.
- **Database:** MySQL – for storing patient profiles, prescriptions, lab results, medicine stock, logs and others information.
- **Security and Authentication:** Secure login via university credentials and Multi-Factor Authentication (MFA) will be implement later.

2.2 Productive Functions

Dishari provides a comprehensive set of functionalities for patients, doctors, dispensers, lab staff, and administrators. The major product functions are below:

2.2.1 Patient Module:

- Secure login via university credentials (ID/Roll) and multi-factor authentication will be implement later.
- Profile management: contact information, session, department, blood group, allergies, and other medical information.
- Prescription and lab report management: view, download, and track current and past prescriptions.
- Upload external test results with automatic notification to the doctor.
- Check real-time availability of lab tests.
- Receive notifications for new prescriptions or updates.
- Rate doctors and dispensers publicly.
- View all staff members and the university ambulance contact.

2.2.2 Doctor Module:

- Login via secure credentials created by admin.
- View and manage patient records and lab results.
- Create, save, and print prescriptions using templates.
- Handle emergency cases for external patients (without dispensing medicines).
- Update or remove irrelevant/unreadable test results.

2.2.3 Dispenser Module:

- Login with role-based access.
- Search prescriptions by patient ID.
- Check medicine stock availability and suggest alternatives if out of stock.
- Input and dispense medicine quantities, with automatic inventory updates.

- Maintain dispensing logs including who dispensed, when, and what medicines.

2.2.4 Lab Module:

- Login with role-based access.
- Accept test requests from internal and external users.
- Process samples and record test results.
- Notify doctors and patients of results.
- Provide printable versions of test reports.

2.2.5 Administrative Module:

- **Manage all users:** add, remove, or modify roles for doctors, dispensers, lab staff, and patients.
- **Manage medicine inventory:** add new items, track stock, and monitor usage.
- **Generate reports:** on prescriptions, medicine usage, and disease trends.
- **Track system history and changes:** who made changes and when.
- **Manage staff rostering:** and shifts, with approvals for duty changes.

3. Stake Holders and Characteristics

3.1 Primary Stakeholders

These are individuals or groups directly involved in the development, maintenance, or operation of the Medical Center.

3.1.1 Project Owner / University Authority

Description:

The project owner, typically represented by the university authority, initiates and funds the Dishari project. They define the core objectives and approve all deliverables to ensure the system aligns with university healthcare needs.

Needs:

- Clear visibility into project progress and milestones.
- Assurance that system meets defined objectives.
- Timely updates and reports on development.

Responsibilities:

- Approve system features and deliverables.
- Provide resources and funding for development.
- Define strategic goals for the medical system.

3.1.2 Development Team

Description:

The development team is responsible for designing, coding, integrating, and deploying the mobile app and backend for Dishari. They ensure the system meets all functional and non-functional requirements.

Needs:

- Detailed specifications and requirements.
- Access to development tools and environments.
- Clear communication with stakeholders.

Responsibilities:

- Implement all required features correctly.
- Integrate frontend and backend modules.
- Ensure code quality and system reliability.

3.1.3 UI/UX Designers

Description:

The UI/UX designers create the mobile app interface using Flutter components, ensuring a consistent, accessible, and user-friendly design across all devices.

Needs:

- Guidelines on branding and user requirements.
- Feedback from internal stakeholders and users.

Responsibilities:

- Design intuitive interfaces for patients, doctors, and staff.
- Maintain consistency and accessibility standards.
- Collaborate with developers to implement designs.

3.1.4 QA Testers

Description:

QA testers validate all features through manual and automated testing to ensure the system is bug-free, secure, and performs reliably.

Needs:

- Access to test environments and scenarios.
- Complete functional and non-functional requirements.

Responsibilities:

- Conduct thorough testing of all modules.

- Report bugs and verify fixes.
- Ensure security, performance, and usability standards.

3.1.5 System Administrator / DevOps Team

Description:

System administrators manage server environments, databases, and deployment pipelines to ensure the uptime, performance, and security of Dishari.

Needs:

- Secure and stable server infrastructure.
- Monitoring tools for performance and errors.

Responsibilities:

- Deploy updates and maintain the system.
- Monitor server health and security.
- Ensure smooth operation of backend services.

3.2 Secondary Stakeholders

3.2.1 Patients (Students/Faculty/Others)

Description:

Patients are the primary users of Dishari. They access their profiles, view prescriptions, upload test results, and receive notifications.

Needs:

- Secure and easy access to their medical records.
- Timely updates on prescriptions and lab reports.
- A user-friendly interface for smooth interaction.

Responsibilities:

- Maintain accurate personal and medical information.
- Upload external test results when needed.
- Provide feedback for system improvement.

3.2.2 Doctors

Description:

Doctors manage patient prescriptions, view lab reports, and handle emergency cases within the system.

Needs:

- Access to up-to-date patient records.

- Tools to create and update prescriptions efficiently.

Responsibilities:

- Ensure accurate prescriptions and proper patient care.
- Monitor patient health records regularly.
- Respond promptly to patient notifications and lab results.

3.2.3 Dispensers

Description:

Dispensers manage medicine dispensing, check stock availability, and suggest alternatives when required.

Needs:

- Real-time access to prescriptions and medicine inventory.
- Tools to update dispensing logs accurately.

Responsibilities:

- Dispense medicines according to prescriptions.
- Update inventory automatically after dispensing.
- Maintain accurate logs for accountability.

3.2.4 Lab Staff

Description:

Lab staff conduct tests, record results, and notify patients and doctors about the outcomes.

Needs:

- Access to patient test requests and history.
- Tools to upload test results digitally.

Responsibilities:

- Ensure timely and accurate lab results.
- Notify relevant users once tests are completed.
- Maintain integrity and accuracy of lab operations.

3.2.5 Administrators (End-user Admins)

Description:

Administrators oversee all modules of Dishari, including user management, inventory, reports, and rostering.

Needs:

- Complete visibility of system operations.
- Tools to efficiently manage staff, stock, and reports.

Responsibilities:

- Ensure smooth operation of the medical system.
- Monitor all users and system activities.
- Generate operational and analytical reports.

3.3 Stakeholder Matrix:

Interest \ Power	High Interest	Low Interest
High Power	Project Owner (University Authority, Administrators), User	Investors/Sponsor
Low Power	Developer Team, UI/UX Designers, QA Testers	Guest, Marketing

4 Design and Implementation Constraints

To ensure the success of the **Dishari** project, several design and implementation constraints have been defined. These constraints guide the structure of the system, the choice of technologies, and the environment in which the application will operate, ensuring consistency, reliability, and security across development, deployment, and usage. **Dishari** will be deployed as a **mobile application** with a backend server using **Serverpod**. The system is designed to operate in a **secure, scalable, and reliable environment** suitable for university healthcare operations.

4.1 System Architecture

4.1.1 Client-side (Mobile App)

- **Supported Platforms:** Android and iOS (latest versions).
- **Framework:** Flutter – for building cross-platform mobile applications with native-like performance.

4.1.2 Server-side (Backend)

- **Operating System:** Linux-based cloud (Ubuntu 20.04 or later)
- **Backend Framework:** Serverpod – for API management, authentication, and database operations
- **Programming Language:** Dart (used with Serverpod and Flutter)
- **Database:** MySQL – for storing patient profiles, prescriptions, lab results, medicine inventory, and logs

- **Deployment Environment:** Cloud server or university-hosted server with secure HTTPS endpoints
- **Notification Services:**
 - Push notifications for Android/iOS
 - Email notifications via SMTP.

4.2 Development Languages and Tools

4.2.1 Framework

Flutter (Dart):

- Used to develop the cross-platform mobile app.
- Ensures native-like performance on Android and iOS.
- Provides responsive and interactive user interfaces.

Serverpod (Dart):

- Handles backend logic, APIs, authentication, and database interactions.
- Provides a modular, secure, and scalable architecture for the system.
- Ensures secure client-server communication and efficient handling of user requests.

4.2.2 Database System

MySQL:

- Relational database for storing patient profiles, prescriptions, lab reports, medicine inventory, and logs.
- Reliable, and efficient for queries and indexing.

4.2.3 Tools

- **Android Studio** – IDE for Flutter app development and Serverpod backend integration.
- **Git & GitHub** – For version control, team collaboration, and issue tracking.

4.2.4 Platform and Deployment Constraints

- The system will be a **mobile application** for Android and iOS devices.
- Backend will be hosted on **Linux-based cloud servers** or a university-hosted server.
- All client-server communication will be secured using **HTTPS**.

- The backend must provide **RESTful APIs or Serverpod RPC endpoints** for all mobile app interactions.

4.2.5 Security & Compliance Constraints

- User passwords and sensitive data must be **securely hashed and encrypted**.
- The system must allow users to **delete accounts** and **export personal data** to comply with privacy policies.
- **Role-Based Access Control (RBAC)** must be applied, with **Multi-Factor Authentication (MFA)** for admin accounts.
- Input validation and server-side checks must prevent **malicious data submission** or unauthorized access.
- All patient records, prescriptions, and lab results must be handled securely according to university data policies.

4.2.6 Design Constraints

- The mobile app interface must follow **clean and minimalist design principles** for an intuitive user experience.
- Consistent **color schemes and UI components** must be used across all screens.
- The app should adopt a **component-based design**, allowing reuse of forms, buttons, cards, and other widgets.
- Navigation and workflows must be **simple and clear**, enabling users to quickly access prescriptions, lab reports, medicine dispensing, and notifications.

5 User Documentation

5.1 Introduction

This documentation is designed to guide **patients, doctors, dispensers, lab staff, and admins** in using system.

5.1.1 User Roles and Access

- The system uses a **role-based login system**. Each user type has unique features:
- **Patient** → Profile, prescriptions, reports, appointments, ratings.
- **Doctor** → Prescriptions, patient records, emergency cases.
- **Dispenser** → Dispense medicines, manage stock.
- **Lab Staff** → Upload test reports, manage lab workflow.
- **Admin** → Manage users, inventory, rosters, reports.

5.1.2 Patient Use Guide

Secure Login

- Patients create an account and log in using their **University email (ID-roll based)**.
- Password recovery option available.

Profile Management

- **Update personal details:** contact info, session, department, blood group, allergies.
- Maintain accurate medical history for better treatment.

Prescriptions & Reports

- **View & Download** all current and past prescriptions.
- **Access Lab Reports** uploaded by university labs.
- **Upload External Reports** (except X-ray, MRI, CT-scan, Ultrasonogram) → system notifies the doctor.
- **Check Test Availability** → view real-time lab capacity before booking.

Notifications

- **Automatic reminders for:** New prescriptions uploaded and Lab test results available.

Ratings & Info

- Rate doctors and dispensers (ratings visible to all users).
- View list of all staff members and access **University Ambulance number**.

5.1.3 Doctor Use Guide

Login & Profile

- Admin creates doctor accounts (ID/email + password).
- Doctors log in with their assigned credentials.

Prescription Management

- Enter **Patient ID**:
 - If record exists → fetch details.
 - If not → system creates a new record.
- Use **prescription templates** for faster entry.
- Save prescriptions → automatically sent to dispensers and patients.
- Print prescriptions when required.

Test Reports & Updates

- Access lab reports directly in the system.
- Update prescriptions based on new test results.
- Delete irrelevant/unreadable test results.

Emergency Cases

- Doctors can treat patients outside the university in urgent cases.
- No medicines will be dispensed through the system in such cases.

5.1.4 Medicine Dispenser Use Guide

Login

- Admin creates dispenser accounts (ID/email + password).
- Login via role-based system.

Prescription Search

- Search prescriptions by **Patient ID**.

Medicine Availability & Dispensing

- System displays **current stock** for each medicine.
- If medicine unavailable:
- System suggests **alternatives**.
- Dispenser selects alternative → inputs quantity.
- After dispensing, stock is updated automatically.

Logs

- **All dispensing actions are recorded:** Who dispensed, when, what medicines, and how much.

5.1.5 Lab Staff User Guide

Login

- Admin creates lab staff accounts.
- Login via role-based authentication.

Test Process

- Patients (inside/outside university) book tests and pay via university payment gateway.
- Provide sample to lab.

Report Management

- Lab staff enter test results into the system.
- Results are **automatically shared** with doctor and patient.
- Print test report if required.

5.1.6 Admin Use Guide

User Management

- **Add/Remove:** Doctors, Dispensers, Lab Staff, Patients

Inventory Management

- Add new medicines.
- Monitor stock levels.
- Track usage history.

Reports & Analytics

Generate reports on:

- Medicine usage (daily, weekly, monthly, yearly).
- Number of prescriptions.
- Current stock levels.
- Disease trends

History Tracking

- Monitor all changes (who made changes, when, and what).

Rostering

- Manage duty schedules for doctors and staff.
- Handle shift-change requests (morning, afternoon, night).

Support & Help

- Contact university IT helpdesk for login issues.
- Contact healthcare admin for profile or medical record issues.

6. Functional Requirements

6.1. User Registration and Login

FR-1:	Users must be able to register and log into the system.
Description:	Patients can create accounts using their university email (ID-roll). Admin creates accounts for Doctors, Dispensers, Lab Staff, and itself. The system must support role-based login: Patient, Doctor, Dispenser, Lab Staff, Admin. Users must authenticate securely with email and password. Multi-factor authentication can be supported.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin, System.
Priority:	High

6.2. Profile Management

FR-2:	Users should manage their profiles.
Description:	Patients can update contact info, blood group, allergies, department, session, emergency contact. Staff (Doctor, Dispenser, Lab Staff) and Admin can view/update assigned info. Users can update password and profile picture.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin, System.
Priority:	High

6.3. Prescription Management

FR-3	Doctors can create, save, and send prescriptions.
Description:	Doctors enter patient ID to fetch records. If patient ID does not exist, a new patient record is created. Prescriptions can be saved, printed, and sent to the dispenser and patient. Doctors can view test results, update prescriptions, or remove irrelevant/unreadable results. Emergency cases: Doctors can input external patient info (outside university).
Stakeholders:	Doctor, Patient, Dispenser, System.
Priority:	High

6.4. Medicine Dispensing

FR-4	Dispensers manage prescriptions and medicine
-------------	--

Description:	Search prescriptions by patient ID. Check stock availability. If medicine is unavailable, suggest alternatives. Dispensers select alternatives, input quantity, and dispense medicines. Inventory stock automatically updates. Dispensing logs record who dispensed, when, and what quantity.
Stakeholders:	Dispenser, Doctor, Patient, Admin, System
Priority:	High

6.5. Lab Test Management

FR-5	Lab Staff manage lab tests for patients.
Description:	Internal and external patients can book tests. Payments can be made via university gateway for external patients. Lab staff receives test samples, performs tests, uploads results. Results are sent to doctor and patient automatically. Printable version of test results is available.
Stakeholders:	Lab Staff, Patient, Doctor, Admin, System.
Priority:	High.

6.6. External Test Upload

FR-6	Patients can upload external test results.
Description:	Upload results except X-ray, MRI, CT-scan, Ultrasonogram. Doctor automatically notified upon upload. Supports file attachment, date, and test type.
Stakeholders:	Patient, Doctor, System.
Priority:	Medium.

6.7. Notifications

FR-7	Users receive notifications.
Description:	Patients are reminded to update prescriptions or test results. Lab results and prescription updates trigger notifications. Admin can send general announcements. Users can mark notifications as read.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin.
Priority:	Medium.

6.8. Ratings

FR-8	Patients can rate doctors and dispensers.
Description:	Rating includes value (1–5) and optional feedback. Ratings are linked to consultation, dispensing, or lab test context. Ratings are visible to admins and relevant staff.
Stakeholders:	Patient, Doctor, Dispenser, Admin, System
Priority:	Medium

6.9. Admin controls

FR-9	Admin manages users, inventory, and reports.
Description:	Add/remove doctors, dispensers, lab staff, or patients. Monitor, track, and update medicine stock levels. Generate reports: prescription usage (daily, weekly, monthly, yearly), disease trends, inventory status. Track all changes (who changed, when). Manage staff rosters and shifts (morning, afternoon, night).
Stakeholders:	Admin, System.
Priority:	High.

6.10. Staff Rostering

FR-10	Staff shifts managed by Admin.
Description:	Assign staff to morning, afternoon, or night shifts. Staff can request change; Admin approves or rejects. Record scheduled, confirmed, cancelled, completed shifts.
Stakeholders:	Admin, Doctor, Dispenser, Lab Staff, System.
Priority:	Medium.

6.11. Payments

FR-11	System handles payments for lab tests.
Description:	Supports university gateway, card, cash, mobile banking. Linked to test booking ID. Stores transaction ID, payment status, and date.
Stakeholders:	Patient, Lab Staff, Admin, System.
Priority:	Medium.

6.12. Audit & History

FR-12	System tracks all critical actions.
--------------	-------------------------------------

Description:	Log who performed action, on which entity, timestamp, old & new values. Admin can view history of prescriptions, tests, dispensing, and inventory changes.
Stakeholders:	Admin, System.
Priority:	High.

7.Non-Functional Requirements

7.1. Performance

NFR-1	The system should respond to user actions within 3 seconds.
Description:	The system must ensure fast response times for login, viewing prescriptions, accessing lab reports, and checking medicine stock. Heavy operations like report generation must complete within acceptable time (≤ 5 seconds).
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin,
Priority:	High.

7.2. Scalability

NFR-2	The system must handle growing numbers of users and data.
Description:	The database and server should support thousands of students, staff, and test records without significant degradation in performance. Future scaling should be possible with minimal architectural changes.
Stakeholders:	Developer, System, Admin.
Priority:	High.

7.3. Security

NFR-3	The system must ensure secure authentication and data protection.
Description:	Role-based access must strictly control permissions. All sensitive data (passwords, test results, prescriptions) must be encrypted in storage and transmission. University ID/email must be validated during patient account creation. Unauthorized access must be prevented.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin, Developer, System.
Priority:	High.

7.4. Availability & Reliability

NFR-4	The system should be available at least 99% of the time.
Description:	The system must minimize downtime. In case of server failure, backup and recovery mechanisms should ensure that patients and staff do not lose access to prescriptions, test reports, or stock data.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin, System.
Priority:	High

7.5. Usability

NFR-5	The system must provide a simple and user-friendly interface.
Description:	Patients must be able to log in and access prescriptions/reports with minimal training. Doctors, Dispensers, and Lab Staff should easily manage their tasks (e.g., prescriptions, test results, dispensing) without confusion. Notifications should be clear and actionable.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin.
Priority:	High

7.6. Maintainability & Extensibility

NFR-6	The system should allow for easy updates and integration of new features.
Description:	The architecture must support modular updates, such as adding new lab tests, new medicine types, or new reporting features in the future without major changes to the core system.
Stakeholders:	Developer, System
Priority:	Medium

7.7. Auditability & Compliance

NFR-7	The system must record all important activities for accountability.
Description:	Audit logs must store information on who performed an action, when, and what changes were made. The system must comply with university data policies and healthcare data handling best practices.

Stakeholders:	Admin, Developer, System
Priority:	High

7.8. Portability

NFR-8	The system should be accessible on multiple platforms.
Description:	The Flutter app must run smoothly on Android (via Android Studio) and optionally.
Stakeholders:	Patient, Doctor, Dispenser, Lab Staff, Admin.
Priority:	Medium

8.Data Requirements

The Medical Center system requires several categories of data to operate effectively. These requirements ensure that all modules (Patient, Doctor, Dispenser, Lab, and Admin) function reliably and securely.

8.1. User Data

- Full Name.
- University ID / Roll (for students & staff).
- Role (Patient, Doctor, Dispenser, Lab Staff, Admin).
- Contact Information (Phone Number, Email).
- Department / Session (for students).
- Password (encrypted for authentication).
- **Stakeholders:** Patients, Doctors, Dispensers, Lab Staff, Administrators.
- **Priority:** High.

8.2. Patient Medical Profile Data

- Blood Group.
- Allergies (if any).
- Medical History (basic details relevant for prescriptions).
- Uploaded External Test Results (PDF, images – except X-ray, MRI, CT-scan, Ultrasonogram).
- **Stakeholders:** Patients, Doctors.
- **Priority:** High.

8.3 Prescription Data

- Patient ID.
- Doctor ID.
- Date and Time of Prescription.
- List of Medicines Prescribed (with dosage and duration).
- Diagnosis / Notes.
- **Stakeholders:** Doctors, Patients, Dispensers.
- **Priority:** High.

8.4 Medicine Inventory Data

- Medicine ID.
- Medicine Name.
- Stock Quantity.
- Expiry Date.
- Alternative Medicines (if applicable).
- Dispensing Logs (who dispensed, when, quantity).
- **Stakeholders:** Dispensers, Administrators.
- **Priority:** High.

8.5 Lab Test Data

- Test Request ID.
- Patient ID.
- Test Type (e.g., Blood, Urine, etc.).
- Payment Receipt / Transaction ID.
- Test Results (digital record + printable version).
- Date and Time of Result Submission.
- **Stakeholders:** Lab Staff, Doctors, Patients.
- **Priority:** High.

8.6 Doctor Data

- Doctor ID.
- Name and Specialization.
- Availability Schedule.
- Emergency Case Records (for external patients).
- **Stakeholders:** Patients, Doctors, Administrators.
- **Priority:** High.

8.7 Notification Data

- Notification Type (e.g., New Prescription, Lab Result, lab test update Reminder).
- Recipient (Patient ID, Doctor ID, Staff ID).
- Timestamp.
- Delivery Method (Push Notification, Email).
- **Stakeholders:** Patients, Doctors, Dispensers, Lab Staff.
- **Priority:** High.

8.8 Feedback & Rating Data

- Patient ID.
- Date and Time.
- Doctor/Dispenser ID being rated.
- Rating (numeric scale).
- Comments/Feedback.
- **Stakeholders:** Patients, Doctors, Dispensers, Administrators.
- **Priority:** Medium.

8.9 Admin & System Management Data

- User Accounts (creation, modification, removal).
- Rostering Information (doctor/staff duty shifts).
- Reports & Analytics (medicine usage, disease trends, test statistics).
- System History / Audit Logs (who made changes, when, what was changed).
- **Stakeholders:** Administrators.
- **Priority:** High.

8.10 System Logs

- Timestamp.
- Type of Activity (e.g., prescription created, medicine dispensed, test result uploaded).
- User ID or Patient ID involved.
- Status / Outcome (success, failure).
- **Stakeholders:** Administrators.
- **Priority:** High.

9. Requirement Prioritization

We used the **MoSCoW prioritization technique** to categorize each requirement into:

- **M** – Must have.
- **S** – Should have.
- **C** – Could have.
- **W** – Won't have (at this time).

9.1 Prioritization Table

ID	Requirement Summary	Category	MoSCoW Priority
FR-1	User Registration and Login.	Functional	Must
FR-2	Patient Profile Management.	Functional	Must
FR-3	Prescription Access (View, Download).	Functional	Must
FR-4	Report Management (Lab & External Reports Upload).	Functional	Must
FR-5	Notifications for Patients (Prescriptions/Results).	Functional	Should
FR-6	Rating Doctors & Dispensers.	Functional	Could
FR-7	Doctor Appointment & Prescription Management.	Functional	Must
FR-8	Medicine Dispensing & Stock Update.	Functional	Must
FR-9	Lab Test Booking, Processing & Results.	Functional	Must
FR-10	Payment via University Gateway	Functional	Must
FR-11	Admin Controls (User, Inventory, Rostering)	Functional	Must
FR-12	History & Audit Logs	Functional	Should
NFR-1	Performance	Non-Functional	Must
NFR-2	Scalability	Non-Functional	Must
NFR-3	Security	Non-Functional	Must
NFR-4	Availability & Reliability	Non-Functional	Must
NFR-5	Usability (UI/UX)	Non-Functional	Should
NFR-6	Maintainability & Extensibility	Non-Functional	Should
NFR-7	Auditability & Compliance	Non-Functional	Must
NFR-8	Portability (Flutter Android & Web)	Non-Functional	Should

9.2 Requirement Engineering Process

Requirements Engineering (RE) determines software requirements according to customer needs. For the University Medical Center System, RE ensures that the requirements are well-

defined, realistic, and aligned with the needs of students, doctors, dispensers, lab staff, and administrators.

9.3 Requirement Elicitation Techniques

Requirement elicitation involves identifying, gathering, and refining the needs of different stakeholders (patients, doctors, dispensers, lab staff). For this project, the following elicitation techniques were used.

9.4 Hold Interviews

Interviews were conducted individually and in small groups with stakeholders. These discussions allowed us to focus on specific requirements of the University Medical Center system. The interview questions were structured to cover:

- Goals and objectives of the system.
- Process of using the system (login, profile management, prescriptions, dispensing, lab testing).
- Key features required by patients, doctors, dispensers, and lab staff, admin.
- Current problems in the manual process and how the new system could solve them.
- Views on improving accessibility, usability, and efficiency.

Examples of Interview Focus:

- Patients: Prescription access, lab results, and notifications.
- Doctors: Prescription templates, patient history access, and report management.
- Dispensers: Medicine availability, dispensing logs, and stock update requirements.
- Lab Staff: Test requests, result entry, and report delivery.

9.5 Sample of Requirement Collection

Requirement collection provided insights into the real needs of system stakeholders. This process involved structured interviews, surveys, and prototype demonstrations.

9.5.1. Requirement Collection – Students (Patients)

Methodology:

Interviews and surveys were conducted with **20 students** from different departments and sessions. Data was collected in two phases to refine requirements after initial feedback.

Findings:

- Students want to securely log in using their University credentials.
- High demand for easy access to prescriptions and downloadable lab reports.
- Preference for notifications about prescription updates, lab results, and appointments.

- Interest in being able to upload external test results for doctors to review.
- Expectation of viewing staff details and ambulance contact in emergencies.
- Desire to rate doctors and dispensers for service improvement.

Key Requirements Identified:

- Secure login (multi-factor authentication).
- Patient profile management with medical details.
- Prescription and lab report access (view & download).
- Upload feature for external test results.
- Real-time notifications and reminders.
- Rating and feedback system.

9.5.2. Requirement Collection – Doctors

Participants: 3 doctors from the University Medical Center.

Methodology:

Structured interviews focusing on their daily workflow, prescription handling, and patient interaction.

Findings:

- Need to quickly search and view patient history.
- Requirement for reusable prescription templates.
- Ability to handle emergency external cases (but without dispensing medicines).
- Need to update or remove irrelevant/unreadable lab results.

Key Requirements Identified:

- Role-based login for doctors.
- Prescription creation, saving, and printing.
- Access to patient profiles and lab reports.
- Emergency patient handling (non-university).
- Report validation and updates.

9.5.3. Requirement Collection – Dispensers

Participants: 2 dispensers from the medical center.

Methodology:

Focused interviews on medicine dispensing workflow and inventory issues.

Findings:

- Need for patient ID–based prescription search.
- Require real-time medicine stock visibility.
- Want automated alternative medicine suggestions if an item is unavailable.
- Desire a logging system to track who dispensed what and when.

Key Requirements Identified:

- Role-based login for dispensers.
- Prescription search by ID.
- Medicine availability & alternative suggestions.
- Automated inventory updates.
- Dispensing logs.

9.5.4. Requirement Collection – Lab Staff

Participants: 1 lab tester and 1 lab assistant.

Methodology:

In-depth interviews covering sample testing, reporting, and communication with doctors/patients.

Findings:

- Need for simple entry of test results linked to patient ID.
- Desire to send reports digitally to both doctors and patients.
- Require print options for physical reports.
- Want system integration with university payment gateway for test fees.

Key Requirements Identified:

- Role-based login for lab staff.
- Lab test management (accept requests, enter results).
- Report notifications to patients and doctors.
- Printable reports.
- Payment confirmation integration.

9.6 Requirement Validation

Requirement validation ensures that requirements are correct and reflect the desired system quality. Initially, requirements appeared satisfactory but revealed ambiguities and gaps during detailed analysis.

9.6.1 Walkthrough

We conducted formal walkthroughs of requirement documents with a team including stakeholders, developers, and medical experts. These sessions provided feedback on

completeness, correctness, and consistency of requirements, ensuring they met all stakeholder needs.

9.6.2 Requirement Review

We organized requirement review sessions where stakeholders evaluated documented system features and workflows. This process ensured that all requirements were clear, consistent, and aligned with user expectations.

9.6.3. User Acceptance Testing (UAT)

We designed UAT scenarios where end-users tested the system to verify requirement fulfillment. This process helped validate that the system met user needs and identified any gaps between requirements and implementation.

9.6.4 Requirements Traceability

We maintained comprehensive traceability matrices to verify that all requirements were implemented in the final system. This ensured no requirement was overlooked and helped in impact analysis for requirement changes.

Requirement Traceability Matrix (RTM)

Requirement ID	Requirement Description	Test Case ID	Verification Method
FR-1	Secure User Registration and Login via University credentials (ID/Roll) with MFA	TC-01	Login form testing, MFA validation
FR-2	Patient Profile Management (contact, session, department, blood group, allergies)	TC-02	Edit & save profile verification
FR-3	Prescription Management (view, download, upload external results)	TC-03	Prescription creation, update, download test
FR-4	Lab Test Management (request, result entry, notifications, print reports)	TC-04	Simulated test entry and result validation
FR-5	Medicine Dispensing with stock update and alternative suggestion	TC-05	Stock check, dispensing log verification
FR-6	Doctor Prescription Templates and Emergency Case Handling	TC-06	Prescription template test & emergency workflow
FR-7	Notifications for patients (prescriptions, reports, reminders)	TC-07	Push/email notification testing
FR-8	Feedback & Ratings for doctors and dispensers	TC-08	Rating form submission and public display check

FR-9	Admin User Management (add/remove doctors, dispensers, lab staff, patients)	TC-09	Role creation & permission testing
FR-10	Admin Rostering (staff duty shift management)	TC-10	Shift scheduling and change approval workflow
FR-11	Reports & Analytics (medicine usage, disease trends, stock reports)	TC-11	Report generation & data validation
NFR-1	Performance (system response time < 3 sec)	TC-12	Load & latency benchmarks
NFR-2	Scalability (support 10,000+ students and staff)	TC-13	Simulated multi-user testing
NFR-3	Security (role-based access, encryption, SSL, MFA)	TC-14	Penetration testing, secure API validation
NFR-4	Usability (simple UI for all roles: patient, doctor, staff)	TC-15	UI/UX review & heuristic evaluation
NFR-5	Availability (99% uptime requirement)	TC-16	Server uptime monitoring
NFR-6	Maintainability & Extensibility (Flutter modular code, Serverpod APIs)	TC-17	Code review & modularity analysis

10.UML Diagrams for Dishari

10.1 Class Diagram for Dishari

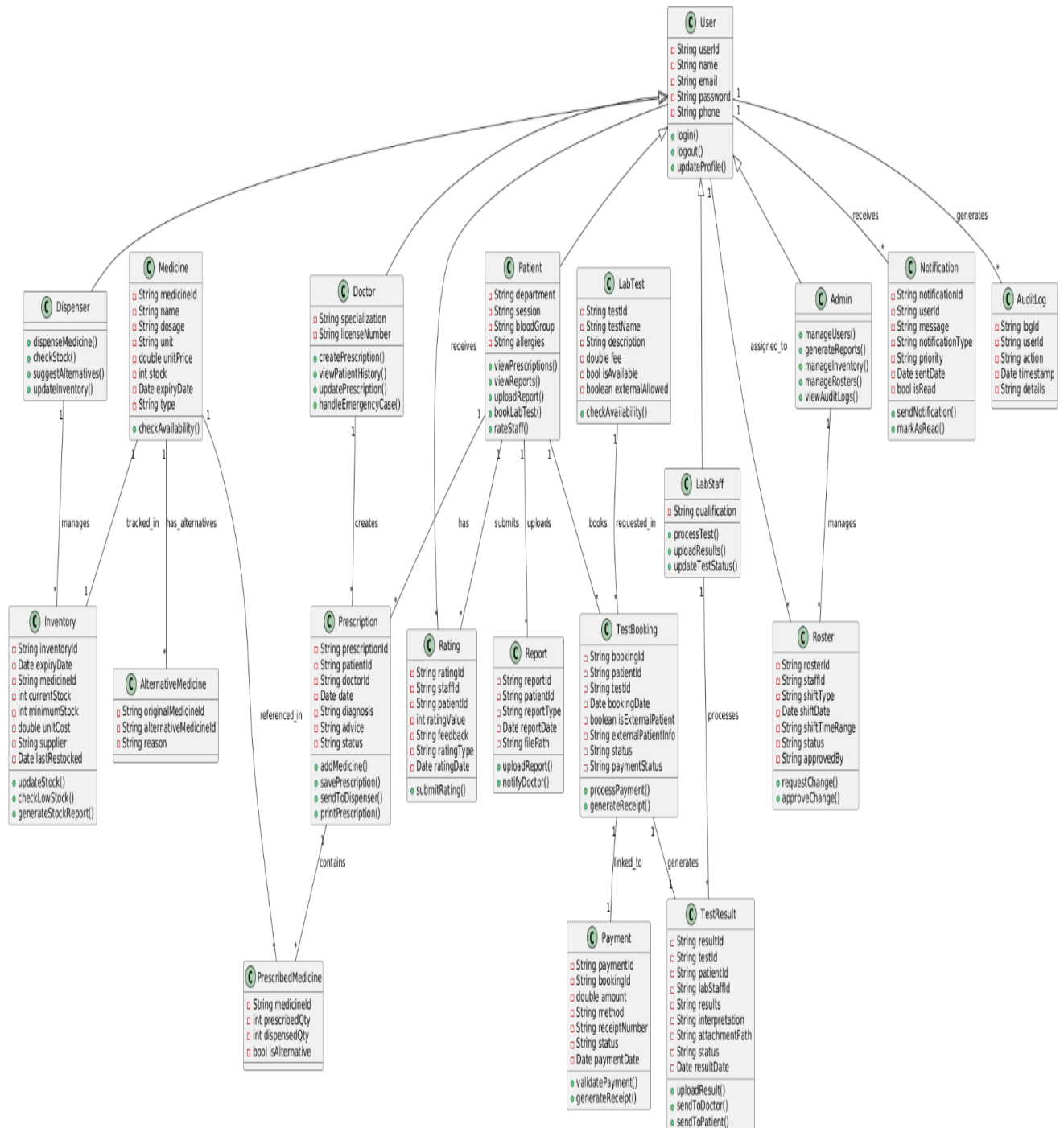


Figure 1: Class Diagram

10.2 Use Case Diagram

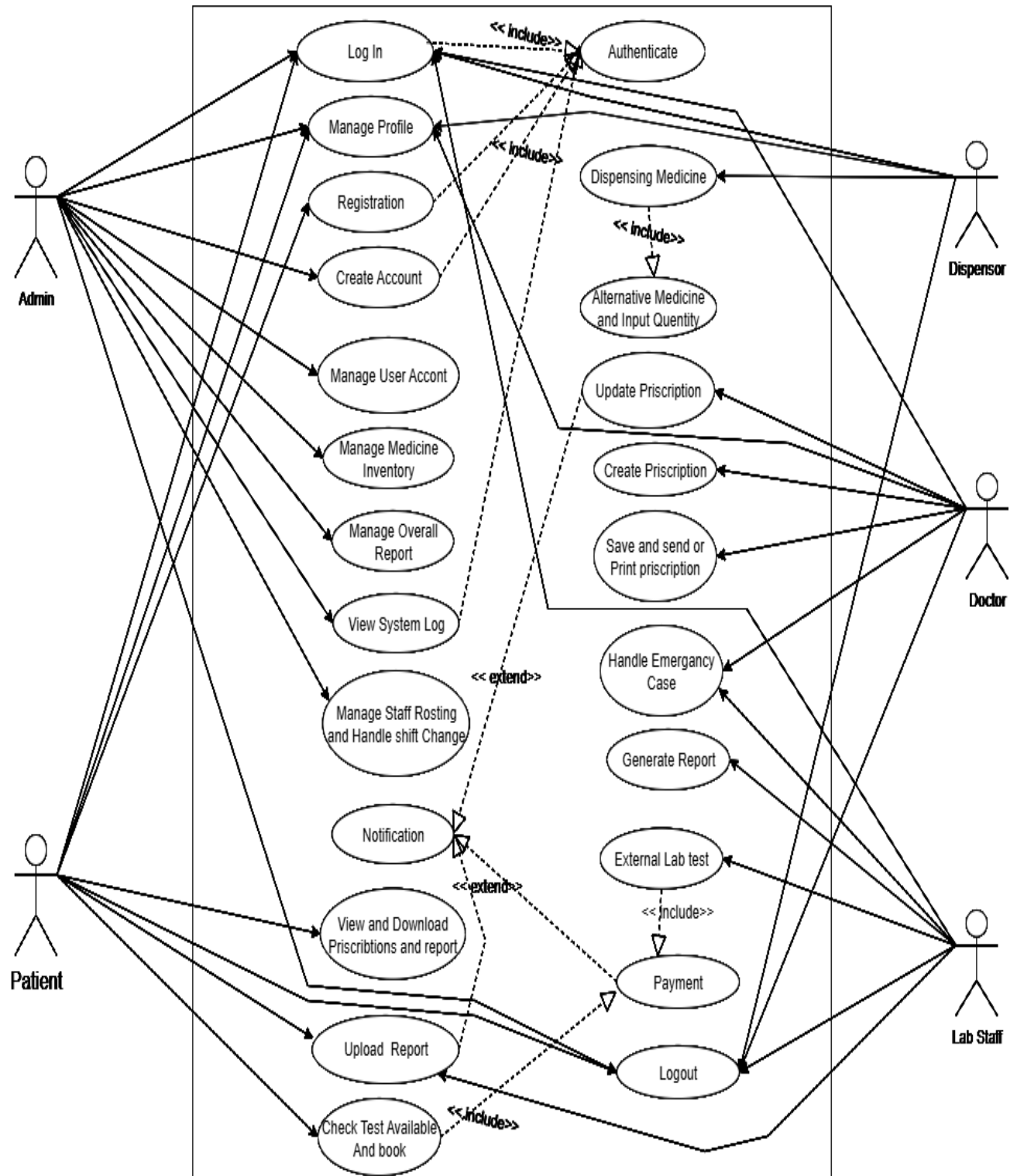


Figure 2: Use Case Diagram

10.3 Use Case description

10.3.1. Patient

Login

Item	Description
Case ID	1.1
Use Case Name	Login
Goal	To authenticate the patient and grant secure access to the system.
Preconditions	Patient has a valid university email/ID and password.
Success End Condition	Login is successful, and the patient dashboard is displayed.
Failed End Condition	Login fails due to invalid credentials or a system error.
Primary Actor(s)	Patient
Secondary Actor(s)	University Authentication Server
Trigger	The patient navigates to the login page and submits their credentials.
Main Success & Alternative Flows	1. Patient enters university ID/email and password. 2. System verifies credentials with the university server. 3. Access is granted, and the dashboard loads. 2a (Alt): Invalid credentials → System displays error. 3a (Alt): System error → Generic error message displayed.
Quality Requirements	Passwords must be encrypted. The session must be secure.

Manage Profile

Item	Description
Case ID	1.2
Use Case Name	Manage Profile
Goal	To allow the patient to view and update their personal information.
Preconditions	Patient is securely logged into the system.
Success End Condition	Profile details are successfully viewed and/or updated.
Failed End Condition	Action fails due to validation or system errors.
Primary Actor(s)	Patient
Secondary Actor(s)	User Profile Database

Trigger	Patient selects the "Manage Profile" option.
Main Success & Alternative Flows	1. Patient requests to view profile. 2. System retrieves and displays details. 3. Patient edits info and saves. 2a (Alt): Database error → Error message. 3a (Alt): Validation error → Invalid fields highlighted.
Quality Requirements	Data must be accurate and private. All changes must be logged.

Registration

Item	Description
Case ID	1.3
Use Case Name	Registration / Create Account
Goal	To create a new patient account in the system.
Preconditions	User has a valid university email ID.
Success End Condition	A new patient account is created successfully.
Failed End Condition	Account creation fails (duplicate ID, system error).
Primary Actor(s)	Patient
Secondary Actor(s)	User Database
Trigger	A new user selects "Register/Create Account".
Main Success & Alternative Flows	1. User provides details (ID, email, etc.). 2. System validates uniqueness. 3. Account is created. 2a (Alt): Duplicate ID → Alert. 3a (Alt): System error → Failure message.
Quality Requirements	Data must be validated. Credentials must be stored securely.

View and Download Prescriptions and Report

Item	Description
Case ID	1.4
Use Case Name	View and Download Prescriptions and Report
Goal	To allow the patient to review and obtain digital copies of prescriptions/reports.
Preconditions	Patient is logged in. Prescriptions/reports are available.
Success End Condition	Documents are displayed and downloaded.

Failed End Condition	Documents cannot be viewed/downloaded.
Primary Actor(s)	Patient
Secondary Actor(s)	Prescription Database, Lab Results Database
Trigger	Patient selects "View Prescriptions" or "View Reports".
Main Success & Alternative Flows	1. Patient requests to view documents. 2. System retrieves and displays list. 3. Patient selects document. 4. System provides file. 2a (Alt): No documents → "No records found". 4a (Alt): Download error → Retry prompt.
Quality Requirements	Documents must be accurate and served securely.

Upload Reports

Item	Description
Case ID	1.5
Use Case Name	Upload Reports
Goal	To allow the patient to submit lab results from external sources.
Preconditions	Patient is logged in. Digital file is available.Except X-ray, MRI, CT-scan, Ultrasonogram
Success End Condition	Report uploaded and linked to patient record.
Failed End Condition	Upload fails (invalid file/system error).
Primary Actor(s)	Patient
Secondary Actor(s)	Lab Results Database
Trigger	Patient selects "Upload Report" and chooses a file.
Main Success & Alternative Flows	1. Patient selects file. 2. System validates file type/size. 3. System stores file & notifies doctor. 2a (Alt): Invalid file → Rejected. 3a (Alt): System error → Upload fails.
Quality Requirements	Files must be validated. Uploads must be secure.

Check Test Available and Book

Item	Description
Case ID	1.6

Use Case Name	Check Test Available and Book
Goal	To check real-time availability of lab tests and book a time slot.
Preconditions	Patient is logged in.
Success End Condition	Availability shown, booking confirmed.
Failed End Condition	Cannot check/book (system error, no slots).
Primary Actor(s)	Patient
Secondary Actor(s)	Lab Scheduling System
Trigger	Patient selects "Book Lab Test".
Main Success & Alternative Flows	1. Patient selects test. 2. System shows slots. 3. Patient books. 4. System confirms. 2a (Alt): No available → Message. 3a (Alt): Choose another.
Quality Requirements	Availability must be real-time. Booking reliable.

Notification

Item	Description
Case ID	1.7
Use Case Name	Notification
Goal	To alert the patient of updates, results, or prescriptions.
Preconditions	Patient is logged in.
Success End Condition	Notification is received and displayed.
Failed End Condition	Notification is not delivered/seen.
Primary Actor(s)	Patient
Secondary Actor(s)	Notification System
Trigger	System generates a new notification.
Main Success & Alternative Flows	1. System sends notification (in-app/email/SMS). 2. Patient views it. 2a (Alt): Delivery error → System retries.
Quality Requirements	Notifications must be secure and prompt.

Logout

Item	Description
Use Case Name	Logout
Goal	To securely end the patient's session and return to the login screen.
Preconditions	Patient is logged in.
Success End Condition	Session is terminated; login screen displayed.
Failed End Condition	Logout fails (system error, session not fully closed).
Primary Actor(s)	Patient
Secondary Actor(s)	Authentication Server
Trigger	Patient clicks the "Logout" button.
Main Success & Alternative Flows	1. Patient initiates logout. 2. System securely terminates session. 3. Login screen displayed. 2a (Alt): System error → Session terminated by timeout.
Quality Requirements	Session must be completely cleared server-side.

10.3.2 Doctor**Create Prescription**

Item	Description
Case ID	2.1
Use Case Name	Create Prescription
Goal	To create a new digital prescription for a patient.
Preconditions	Doctor is logged in, patient selected.
Success End Condition	Prescription created & saved.
Failed End Condition	Not saved (validation/system error).
Primary Actor(s)	Doctor
Secondary Actor(s)	Prescription DB, Patient DB
Trigger	Doctor selects "Create Prescription".

Main Success & Alternative Flows	1. Select template. 2. Enter medicine/dosage/instructions. 3. Save prescription. 4. System stores it. 3a (Alt): Validation error → Highlighted fields. 4a (Alt): Save error → Retry prompt.
Quality Requirements	Data accurate, auto-save drafts.

Update Prescription

Item	Description
Case ID	2.2
Use Case Name	Update Prescription
Goal	Modify existing prescription.
Preconditions	Doctor logged in, prescription open.
Success End Condition	Changes saved & logged.
Failed End Condition	Update fails.
Primary Actor(s)	Doctor
Secondary Actor(s)	Prescription DB
Trigger	Doctor selects "Edit".
Main Success & Alternative Flows	1. Doctor edits prescription. 2. Save changes. 3. System records version history. 2a (Alt): Already dispensed → Warn doctor. 3a (Alt): Save error → Not saved.
Quality Requirements	Maintain change history.

Send & Save / Print Prescription

Item	Description
Case ID	2.3
Use Case Name	Send & Save / Print Prescription
Goal	Finalize prescription, send, and print.
Preconditions	Prescription is open.
Success End Condition	Prescription saved, sent, printed.
Failed End Condition	Save/send/print fails.
Primary Actor(s)	Doctor

Secondary Actor(s)	Prescription DB, Printer, Notification System
Trigger	Doctor clicks "Save & Send/Print".
Main Success & Alternative Flows	1. Save & Send. 2. System saves prescription. 3. Sends copy to dispenser/patient. 4. Prints if selected. 3a (Alt): Not found → Log error 4a (Alt): Printer error → Notify doctor.
Quality Requirements	Copy must be secure (e.g., PDF).

Handle Emergency Cases

Item	Description
Case ID	2.4
Use Case Name	Handle Emergency Cases
Goal	Provide treatment for emergency cases, including outsiders.
Preconditions	Doctor logged in; emergency declared.
Success End Condition	Emergency handled; record created; no medicine dispensed for outside patients.
Failed End Condition	Case not logged; system error.
Primary Actor(s)	Doctor
Secondary Actor(s)	Emergency Case System, Patient Database
Trigger	Patient arrives in emergency.
Main Success & Alternative Flows	1. Select "Handle Emergency". 2. Create temp patient record if needed. 3. Provide treatment; create prescription. 4. Flag prescription to prevent dispensing for outsiders. 2a (Alt): System slow → Bypass full registration.
Quality Requirements	Fast response; humanitarian override possible.

Manage Profile

Item	Description
Case ID	2.5
Use Case Name	Manage Profile
Goal	Allow the doctor to view and update personal and professional information.

Preconditions	Doctor is securely logged into the system.
Success End Condition	Profile details successfully viewed and/or updated.
Failed End Condition	Action fails due to validation or system errors.
Primary Actor(s)	Doctor
Secondary Actor(s)	User Profile Database
Trigger	Doctor selects the "Manage Profile" option.
Main Success & Alternative Flows	1. Doctor requests to view profile. 2. System retrieves and displays current details (contact info, specialization, schedule, etc.). 3. Doctor edits information and saves changes. 2a (Alt): Database error → System displays error message. 3a (Alt): Validation error → Invalid fields highlighted.
Quality Requirements	Data must be accurate, private, and securely stored; all changes logged for auditing.

10.3.3 Lab Test

Lab Test

Item	Description
Case ID	3.1
Use Case Name	External Lab Test
Goal	Process lab test request (internal or external).
Preconditions	Lab staff logged in; sample and payment provided.
Success End Condition	Test processed; results entered into system.
Failed End Condition	Test fails (sample/equipment error).
Primary Actor(s)	Lab Staff
Secondary Actor(s)	Lab Equipment, Lab Results DB
Trigger	New sample logged.
Main Success & Alternative Flows	1. Retrieve sample. 2. Perform test. 3. Enter results. 2a (Alt): Invalid sample → Cancel test; notify patient.
Quality Requirements	Tests performed per strict medical protocols.

Generate Reports (Lab Results)

Item	Description
Case ID	3.2
Use Case Name	Generate Lab Report
Goal	Generate and publish digital lab report.
Preconditions	Test results entered.
Success End Condition	Report generated; doctor and patient notified.
Failed End Condition	Report generation fails.
Primary Actor(s)	Lab Staff
Secondary Actor(s)	Lab Results DB, Notification System
Trigger	Results saved.
Main Success & Alternative Flows	1. Finalize results. 2. Generate PDF report. 3. Notify doctor & patient. 2a (Alt): Formatting error → Alert admin.
Quality Requirements	Reports accurate, legible, and fast.

Manage Profile

Item	Description
Case ID	3.3
Use Case Name	Manage Profile
Goal	Allow the lab staff to view and update personal and professional information.
Preconditions	Lab staff is securely logged into the system.
Success End Condition	Profile details successfully viewed and/or updated.
Failed End Condition	Action fails due to validation or system errors.
Primary Actor(s)	Lab Staff
Secondary Actor(s)	User Profile Database
Trigger	Lab staff selects the "Manage Profile" option.

Main Success & Alternative Flows	1. Lab staff requests to view profile. 2. System retrieves and displays current details (contact info, role, assigned lab sections, etc.). 3. Lab staff edits information and saves changes. 2a (Alt): Database error → System displays error message. 3a (Alt): Validation error → Invalid fields highlighted.
Quality Requirements	Data must be accurate, private, and securely stored; all changes logged for auditing.

10.3.4 Dispenser

Dispense Medicines

Item	Description
Case ID	4.1
Use Case Name	Dispense Medicines
Goal	Process prescription and provide medicine.
Preconditions	Dispenser logged in; valid prescription received.
Success End Condition	Medicine dispensed; inventory updated.
Failed End Condition	Dispensing fails (out of stock/system error).
Primary Actor(s)	Dispenser
Secondary Actor(s)	Prescription DB, Inventory DB
Trigger	New prescription appears in queue.
Main Success & Alternative Flows	1. Select prescription. 2. Show medicine and quantity. 3. Confirm and dispense. 4. System deducts inventory. 3a (Alt): Out of stock → Suggest alternatives (4.3).
Quality Requirements	Inventory update in real-time.

Alternative Medicine and Input Quantity

Item	Description
Case ID	4.2
Use Case Name	Alternative Medicine & Input Quantity
Goal	Suggest and select substitute if prescribed medicine unavailable.

Preconditions	Prescribed medicine unavailable; alternatives exist.
Success End Condition	Alternative selected, dispensed, and logged.
Failed End Condition	No alternative found; process cancelled.
Primary Actor(s)	Dispenser
Secondary Actor(s)	Inventory Database
Trigger	System detects stock-out.
Main Success & Alternative Flows	1. Display alternative medicines. 2. Dispenser selects and inputs quantity. 3. System logs alternative. 2a (Alt): No suitable alternative → Cancel and notify doctor.
Quality Requirements	Alternatives must be medically relevant and approved.

10.3.5 Admin

Login

Item	Description
Case ID	5.1
Use Case Name	Login
Goal	Authenticate admin; grant full system access.
Preconditions	Admin has valid master credentials.
Success End Condition	Logged in; admin dashboard displayed.
Failed End Condition	Login fails.
Primary Actor(s)	Admin
Secondary Actor(s)	Authentication Server
Trigger	Go to login page.
Main Success & Alternative Flows	1. Enter login details. 2. Authenticate. 3. Full access granted. 2a (Alt): Invalid login → Error.
Quality Requirements	Highest security.

Manage User Accounts

Item	Description
Case ID	5.2
Use Case Name	Manage User Accounts
Goal	Create, edit, or deactivate user accounts.
Preconditions	Admin logged in.
Success End Condition	User accounts successfully managed.

Failed End Condition	Action fails (duplicate email/system error).
Primary Actor(s)	Admin
Secondary Actor(s)	User Database
Trigger	Navigate to "User Management".
Main Success & Alternative Flows	1. Select action (Add/Edit/Deactivate). 2. Enter/modify details. 3. System validates and saves. 3a (Alt): Validation error → Prompt correction.
Quality Requirements	All changes logged in audit trail.

Manage Medicine Inventory

Item	Description
Case ID	5.3
Use Case Name	Manage Medicine Inventory
Goal	Add medicines; monitor stock.
Preconditions	Admin logged in.
Success End Condition	Inventory updated; levels viewed.
Failed End Condition	Update fails.
Primary Actor(s)	Admin
Secondary Actor(s)	Inventory Database
Trigger	Navigate to "Inventory Management".
Main Success & Alternative Flows	1. Add/view medicine. 2. Enter details. 3. System adds. 3a (Alt): Medicine exists → Suggest update stock.
Quality Requirements	Inventory accurate; real-time.

Generate Reports (System)

Item	Description
Case ID	5.4
Use Case Name	Generate Reports
Goal	Analytical reports on prescriptions, disease trends, inventory.
Preconditions	Admin logged in; data exists.

Success End Condition	Report generated and displayed.
Failed End Condition	Report fails (no data/system error).
Primary Actor(s)	Admin
Secondary Actor(s)	Various Databases
Trigger	Select "Generate Report".
Main Success & Alternative Flows	1. Select type and date range. 2. System compiles and generates. 3. Display/export. 2a (Alt): No data → Inform admin.
Quality Requirements	Reports fast and accurate.

View System Logs

Item	Description
Case ID	5.5
Use Case Name	View System Logs
Goal	Review system activity and audit trail.
Preconditions	Admin logged in.
Success End Condition	Logs displayed.
Failed End Condition	Logs inaccessible.
Primary Actor(s)	Admin
Secondary Actor(s)	System Log Database
Trigger	Navigate to "System Logs".
Main Success & Alternative Flows	1. Apply filters. 2. Display log entries. 2a (Alt): DB error → Logs unavailable.
Quality Requirements	Logs immutable and tamper-proof.

Manage Staff Posting and Handle Shift Change

Item	Description
Case ID	5.6
Use Case Name	Manage Staff Posting & Shift Change
Goal	Create rosters; handle shift change requests.
Preconditions	Admin logged in.

Success End Condition	Roster updated; requests approved/denied.
Failed End Condition	Action fails (conflict/system error).
Primary Actor(s)	Admin
Secondary Actor(s)	Roster Database
Trigger	Navigate to "Roster Management" / receive notification.
Main Success & Alternative Flows	1. View roster/requests. 2. Make changes/respond. 3. System updates roster & notifies. 2a (Alt): Conflict → Warn before saving.
Quality Requirements	Avoid double-booking/under-staffing.

Logout

Item	Description
Case ID	5.7
Use Case Name	Logout
Goal	Securely end admin session.
Preconditions	Admin logged in.
Success End Condition	Session terminated.
Failed End Condition	Logout fails.
Primary Actor(s)	Admin
Secondary Actor(s)	Authentication Server
Trigger	Click "Logout".
Main Success & Alternative Flows	1. Initiate logout. 2. System ends session. 2a (Alt): Error → Timeout ends session.
Quality Requirements	Session securely cleared.

10.4 Activity Diagram

Patient Registration

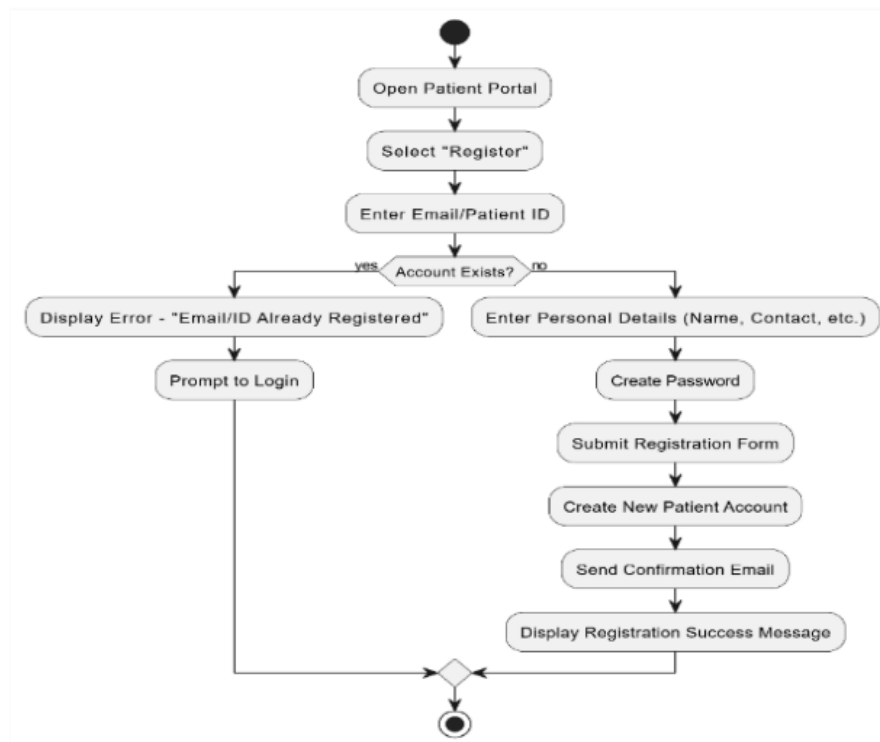


Figure 3: Patient registration

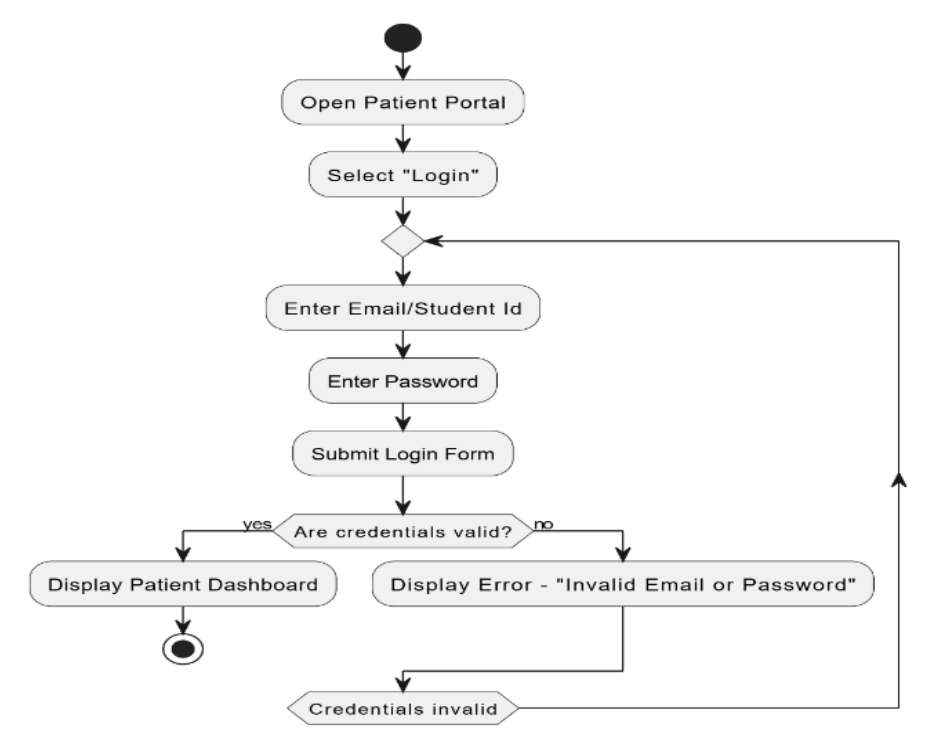


Figure 4: Patient login

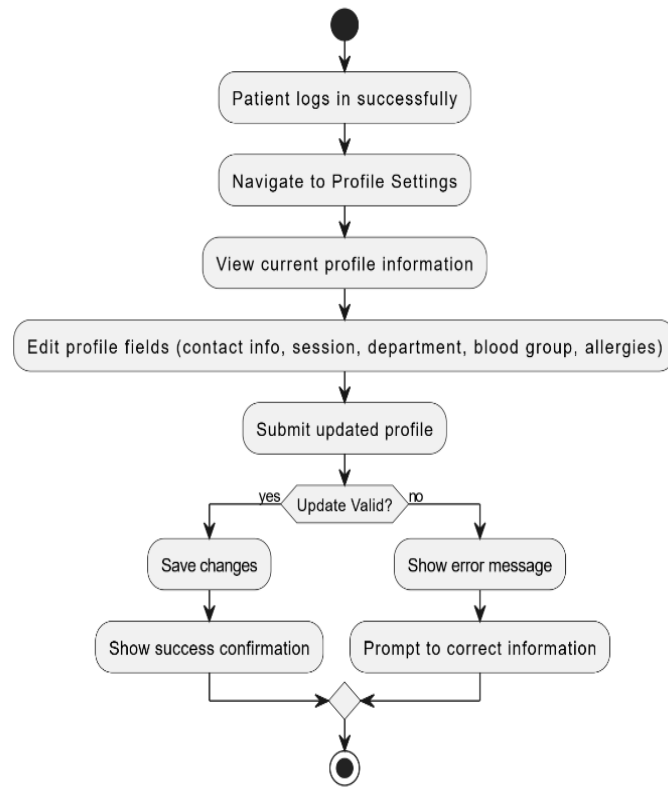


Figure 5: Patient profile update

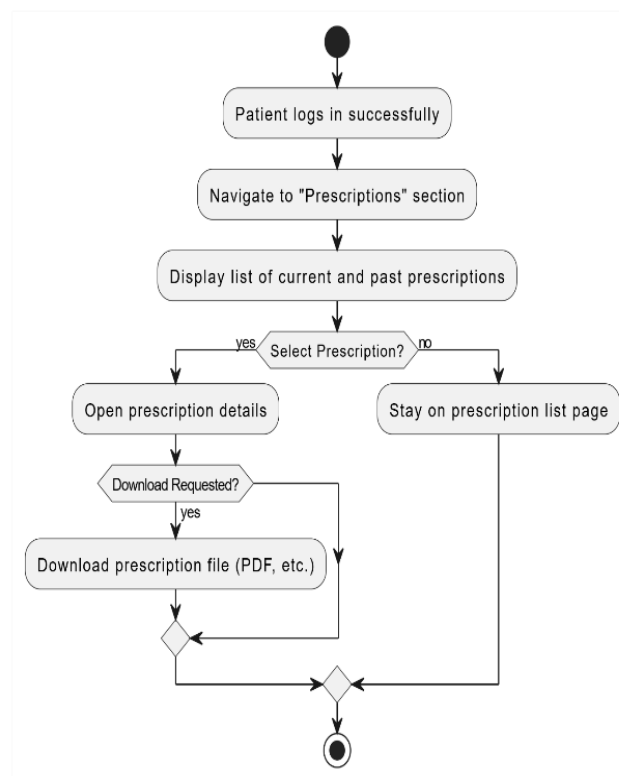


Figure 6: Patient prescriptions (view or download)

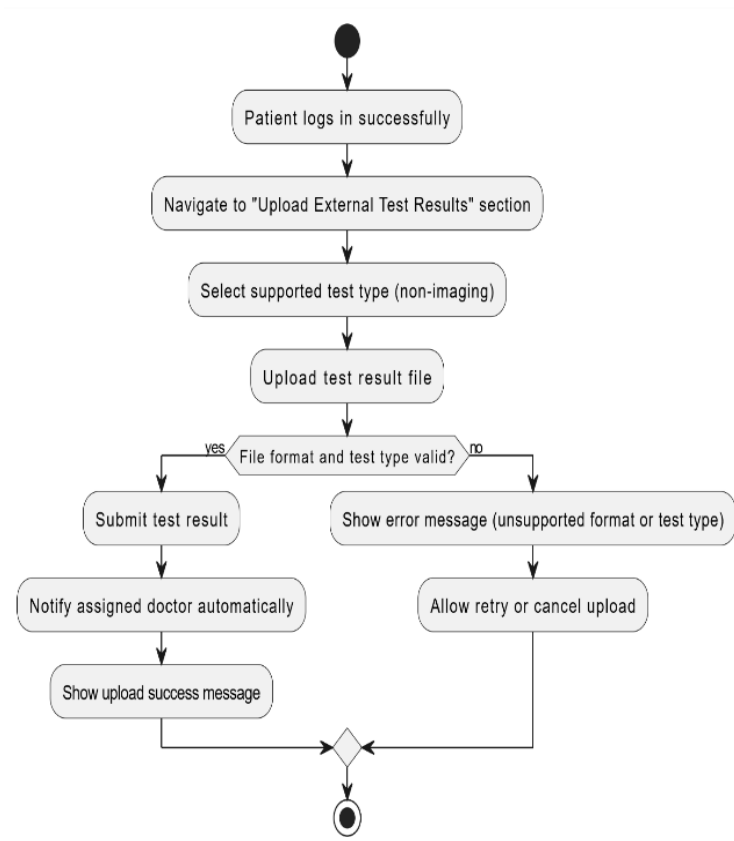


Figure 7: Patient Upload External Test Results

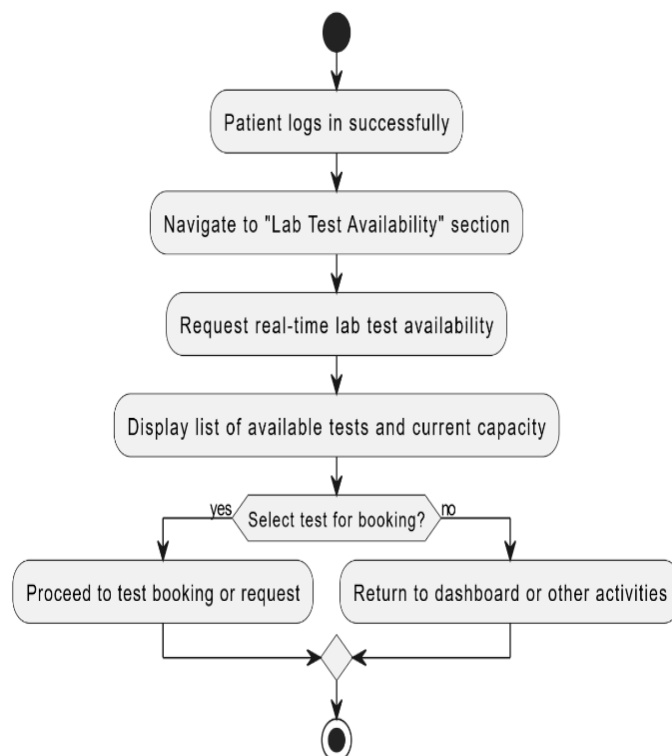


Figure 8: Patient Check Lab Test Availability

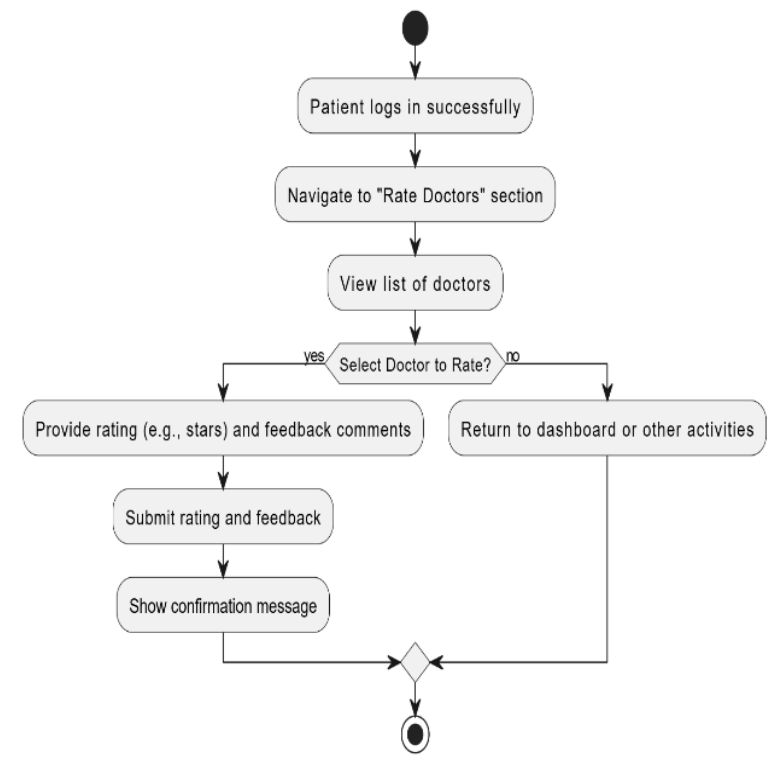


Figure 9: Rate Doctor Same as Others

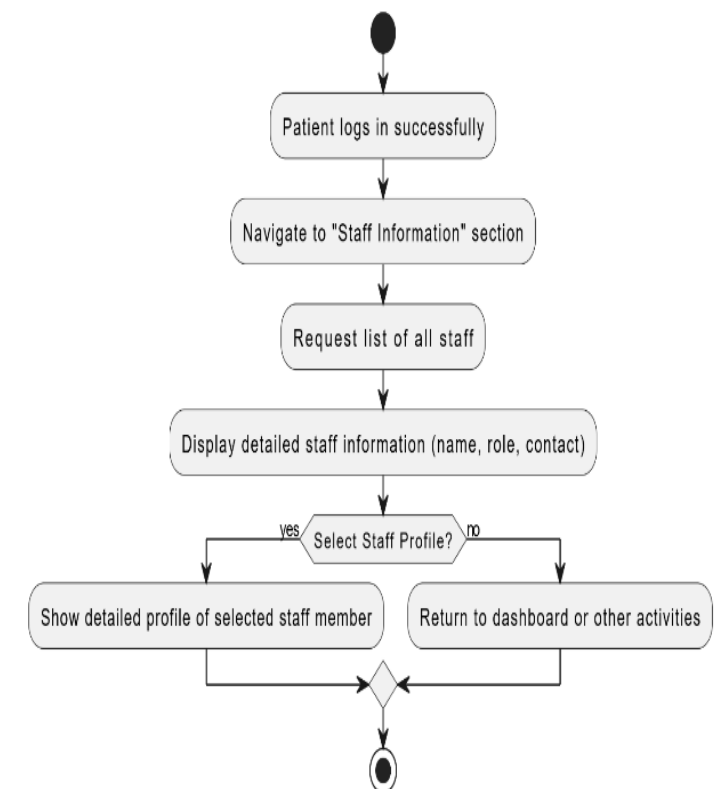


Figure 10: See Staff Information

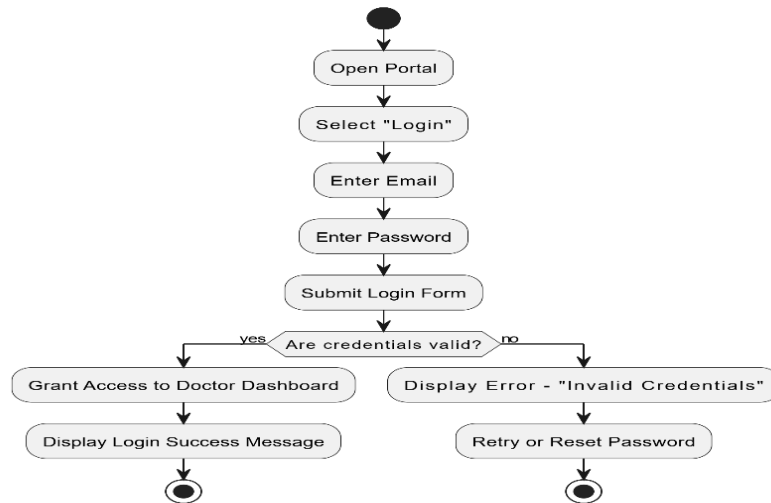


Figure 11: Doctor Log In (Lab and Dispenser login same as Doctor)

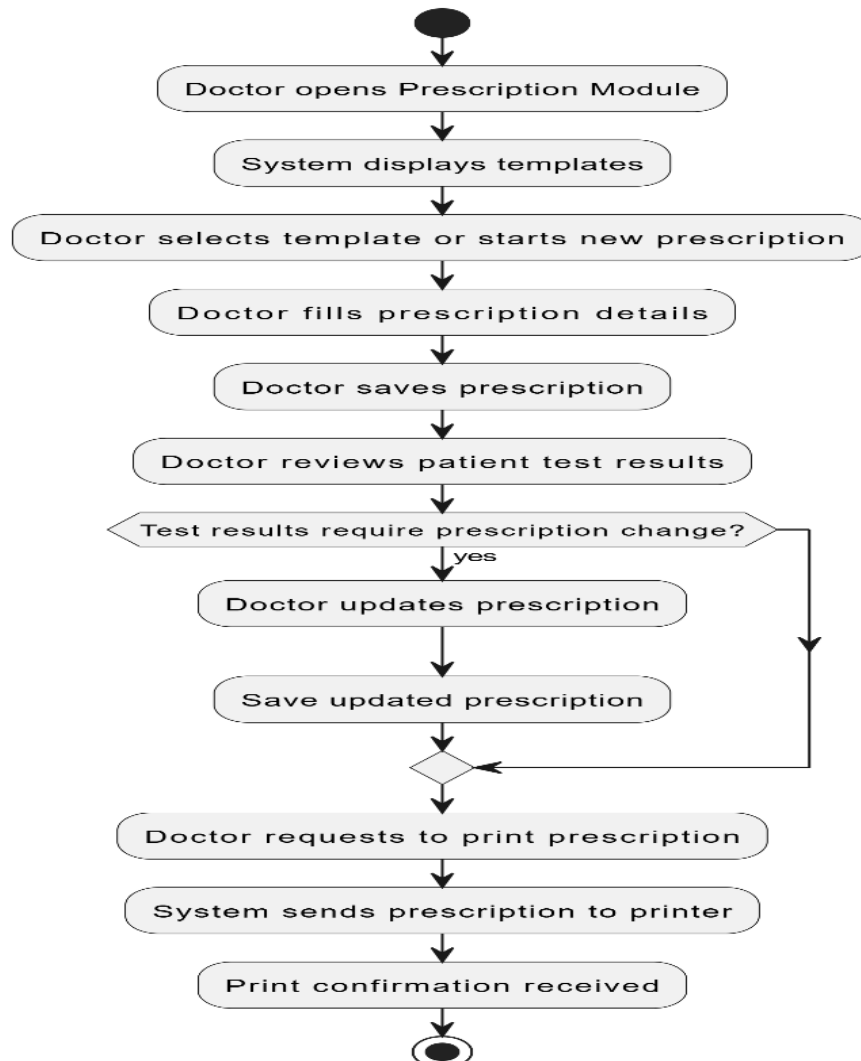


Figure 12: Create/Edit/Print Prescriptions using Templates

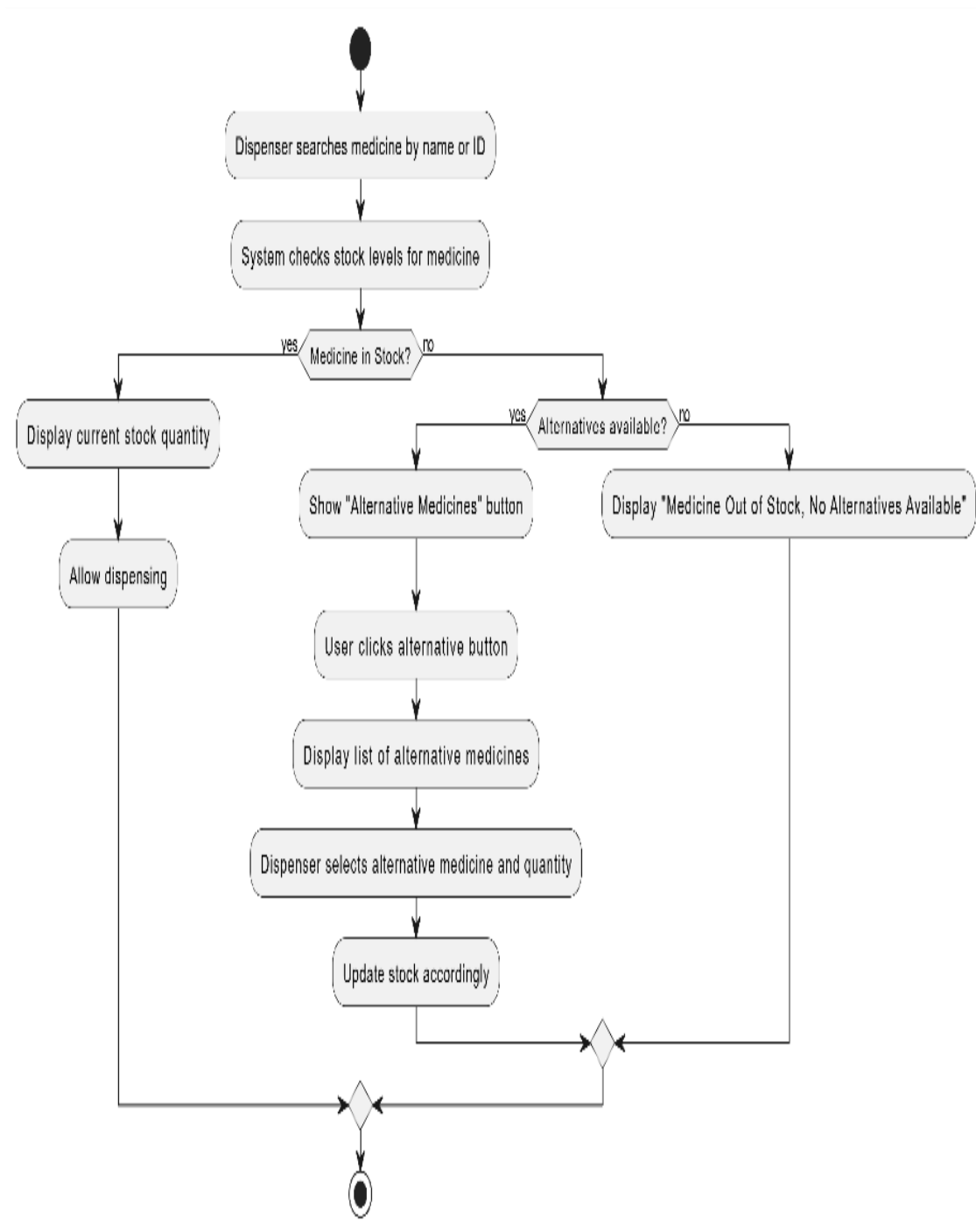


Figure 13: Check Medicine Stock, Suggest Alternatives and dispenser

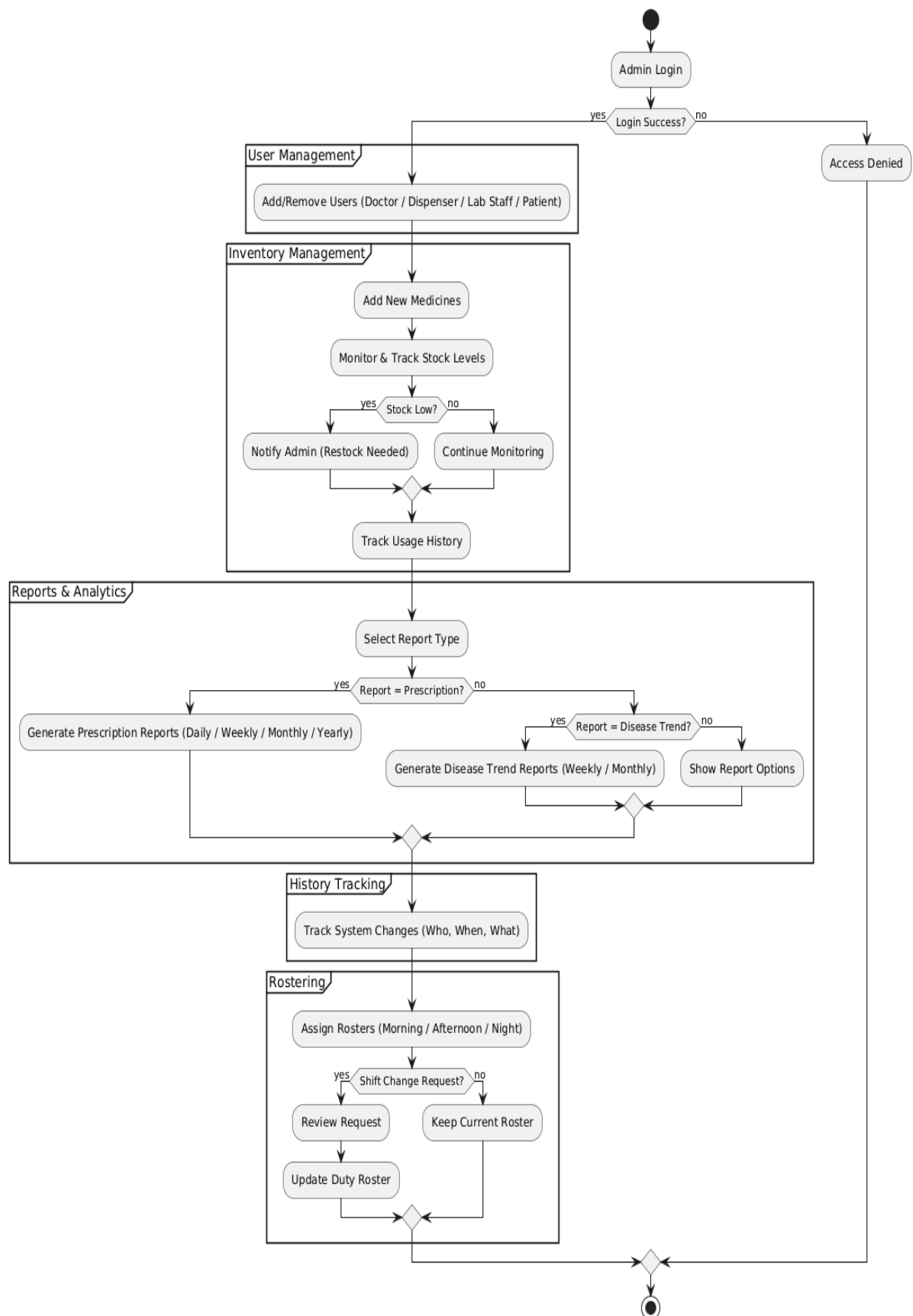


Figure 14: Admin Activity diagram

10.5 Sequence Diagram

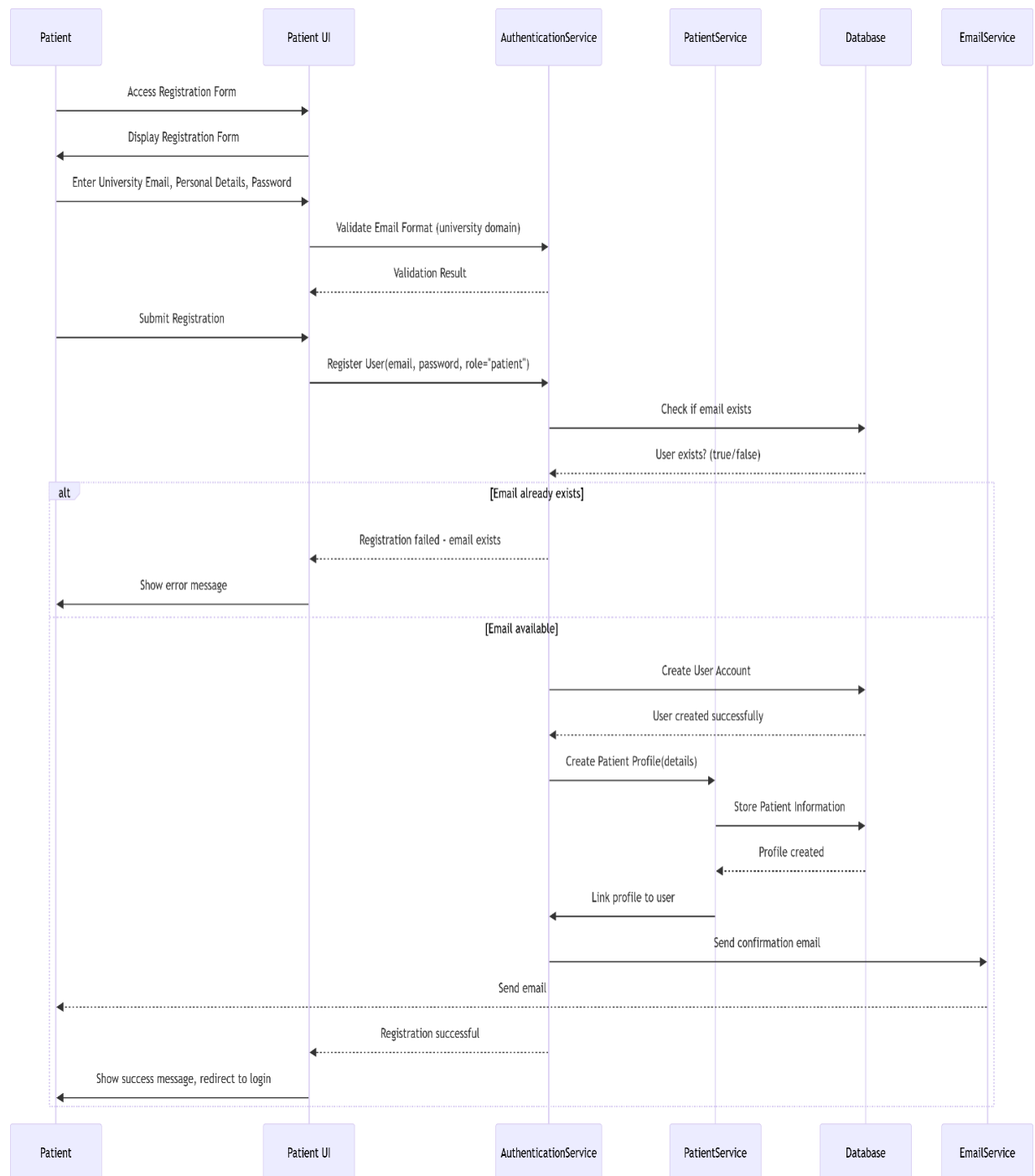


Figure 15: Patient Registration

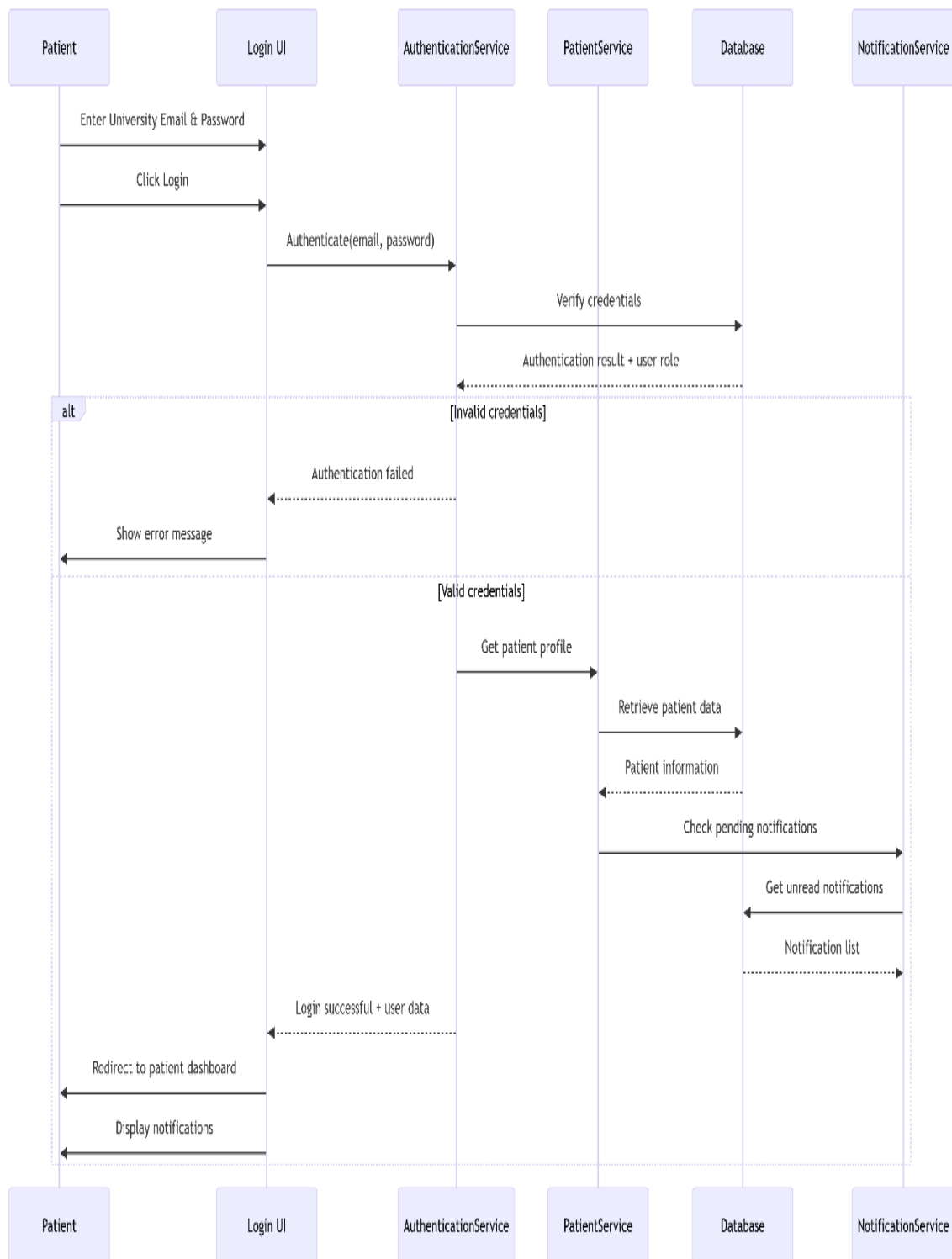


Figure 16: Patient Login

Software Requirements Specification for Dishari

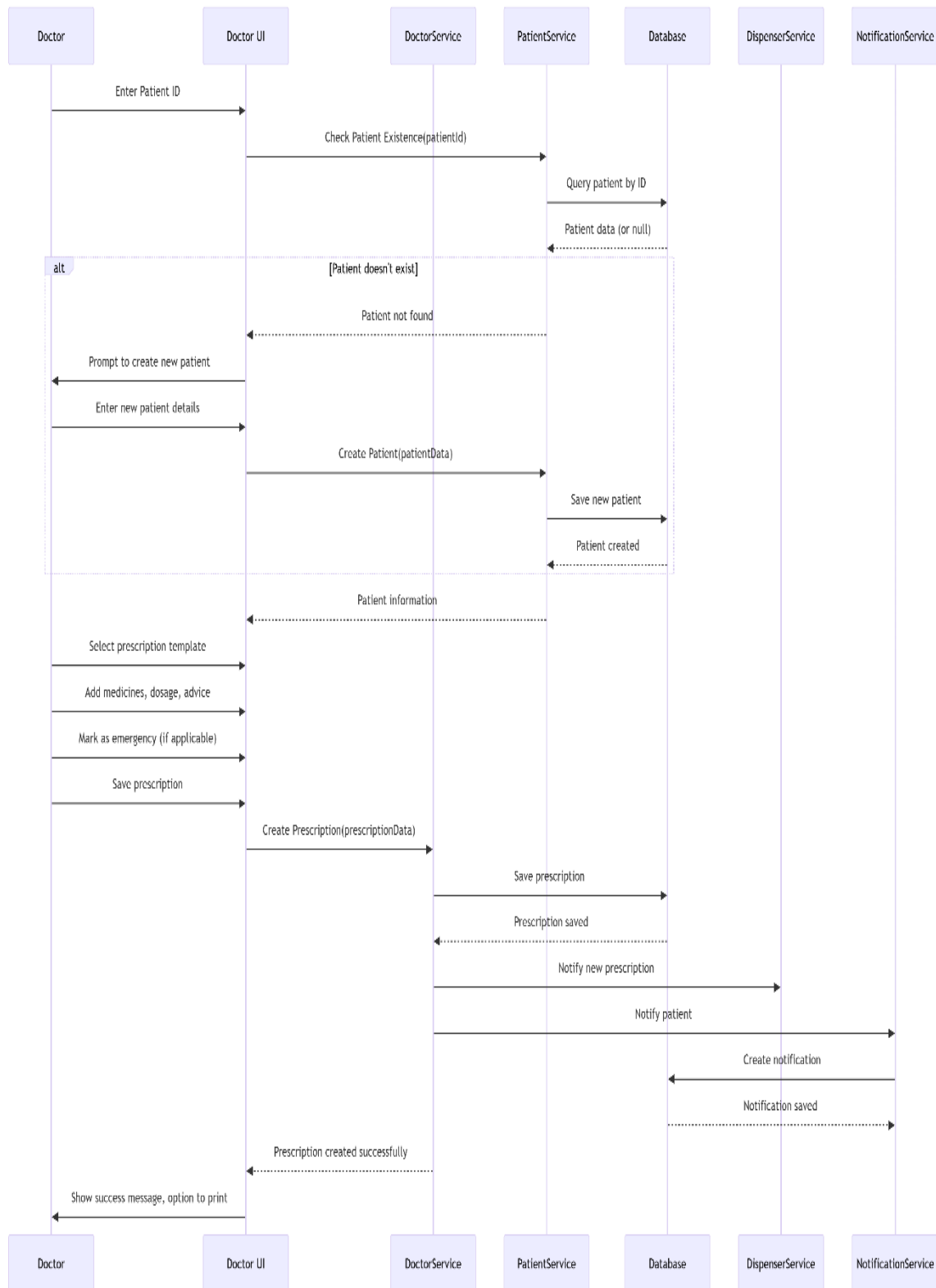


Figure 17: Doctor Creating Prescription

Software Requirements Specification for Dishari

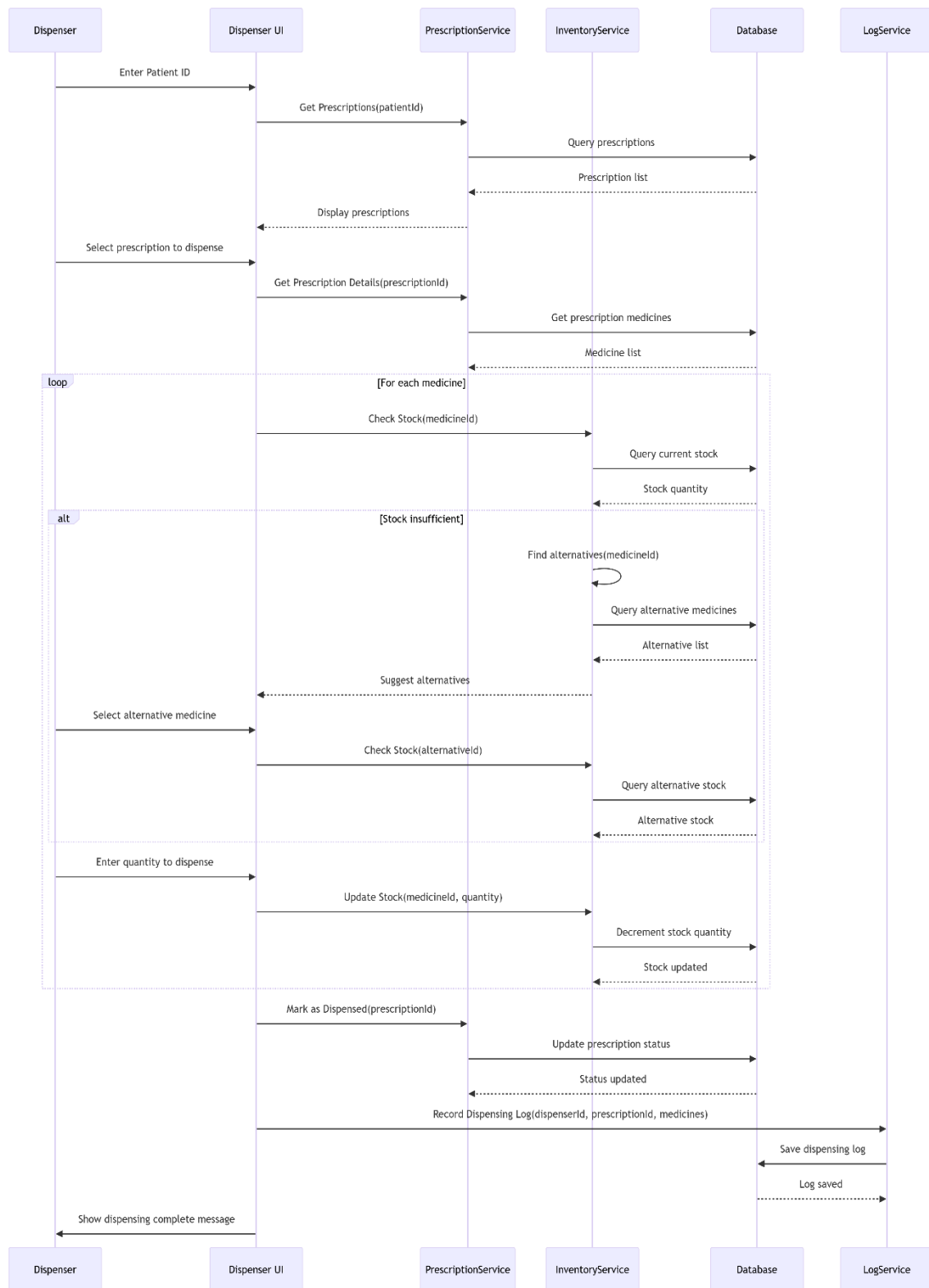


Figure 18: Medicine Dispensing

Software Requirements Specification for Dishari

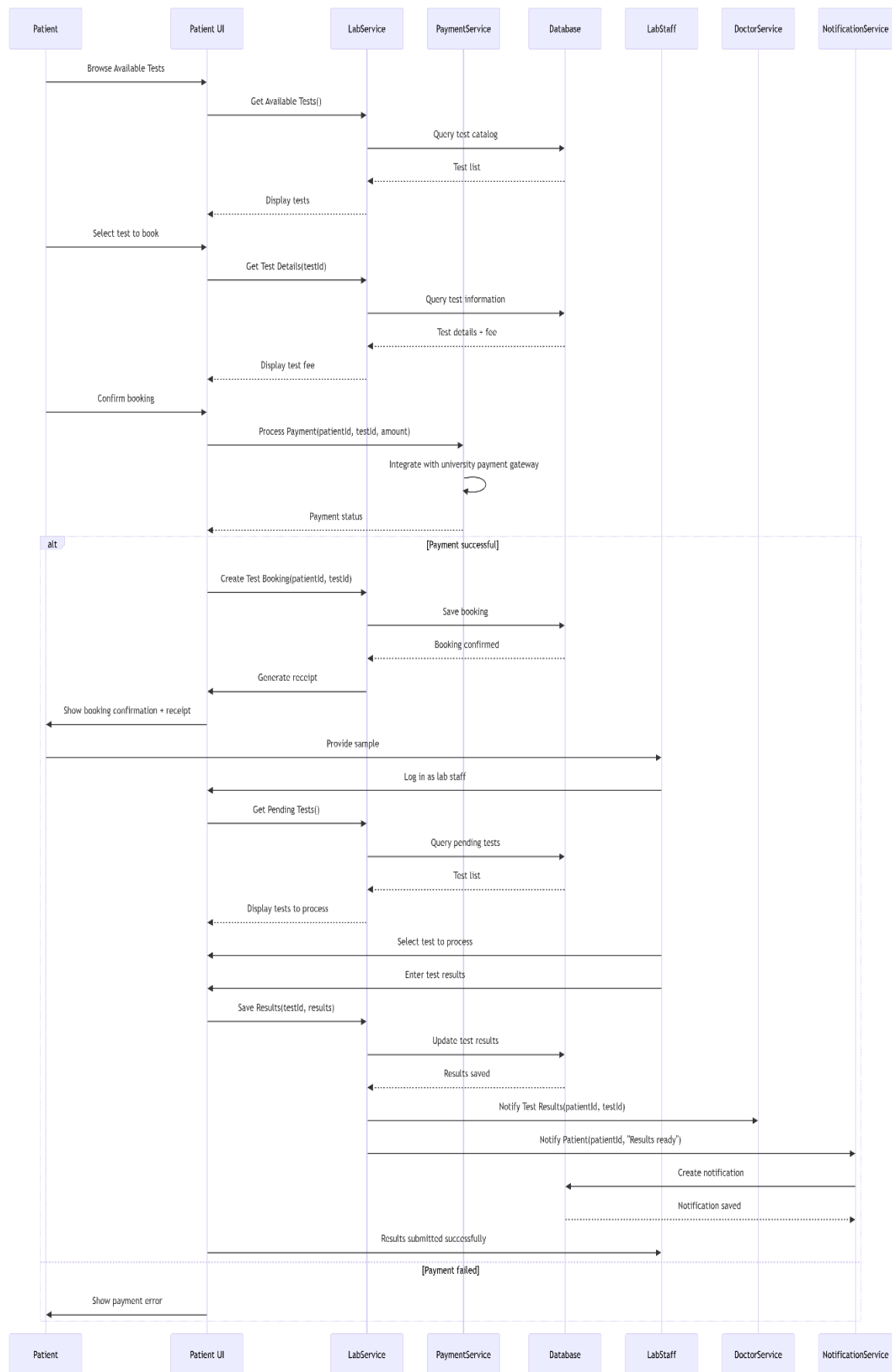


Figure 19: Lab Test Processing

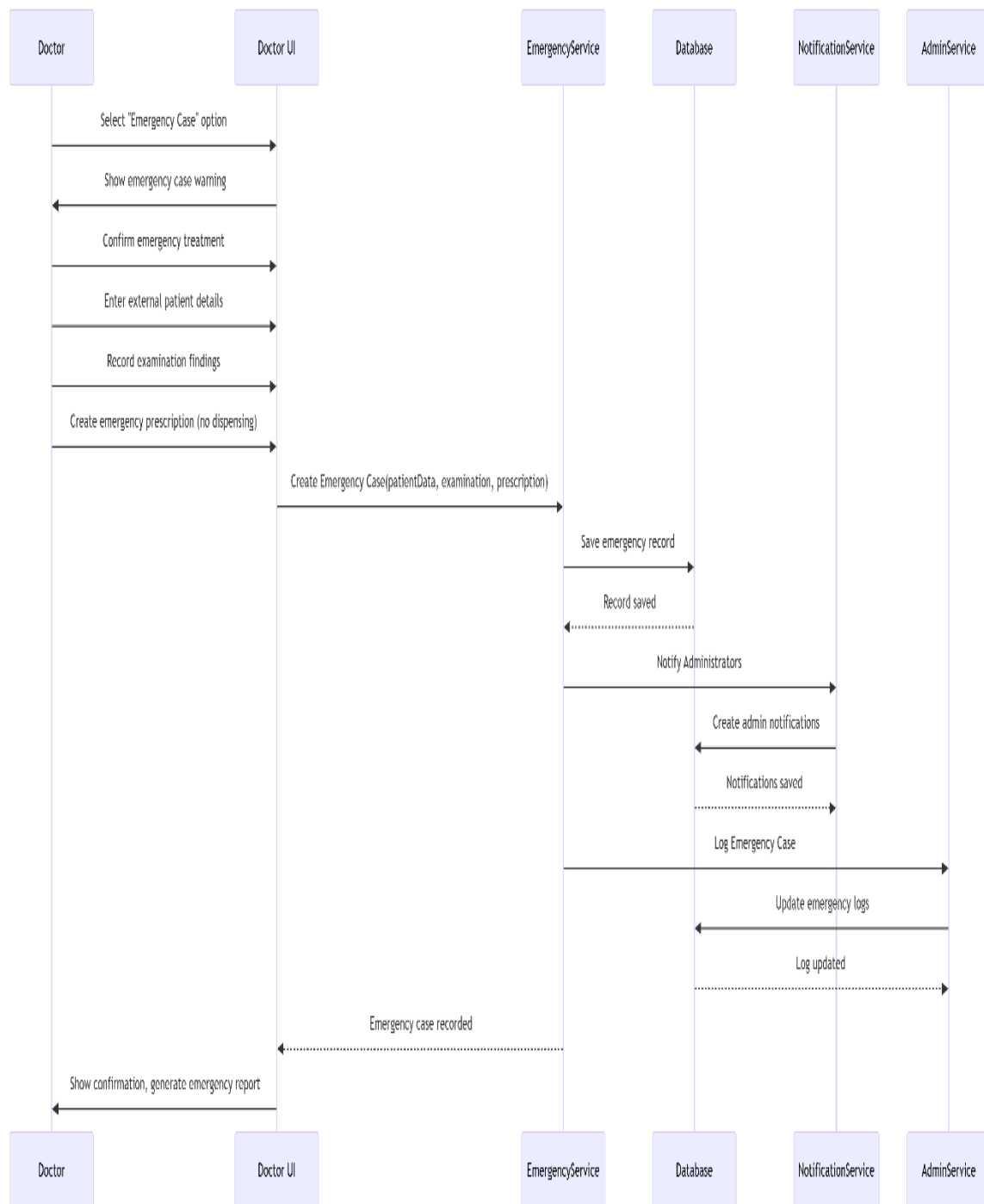


Figure 20: Emergency Case Handling

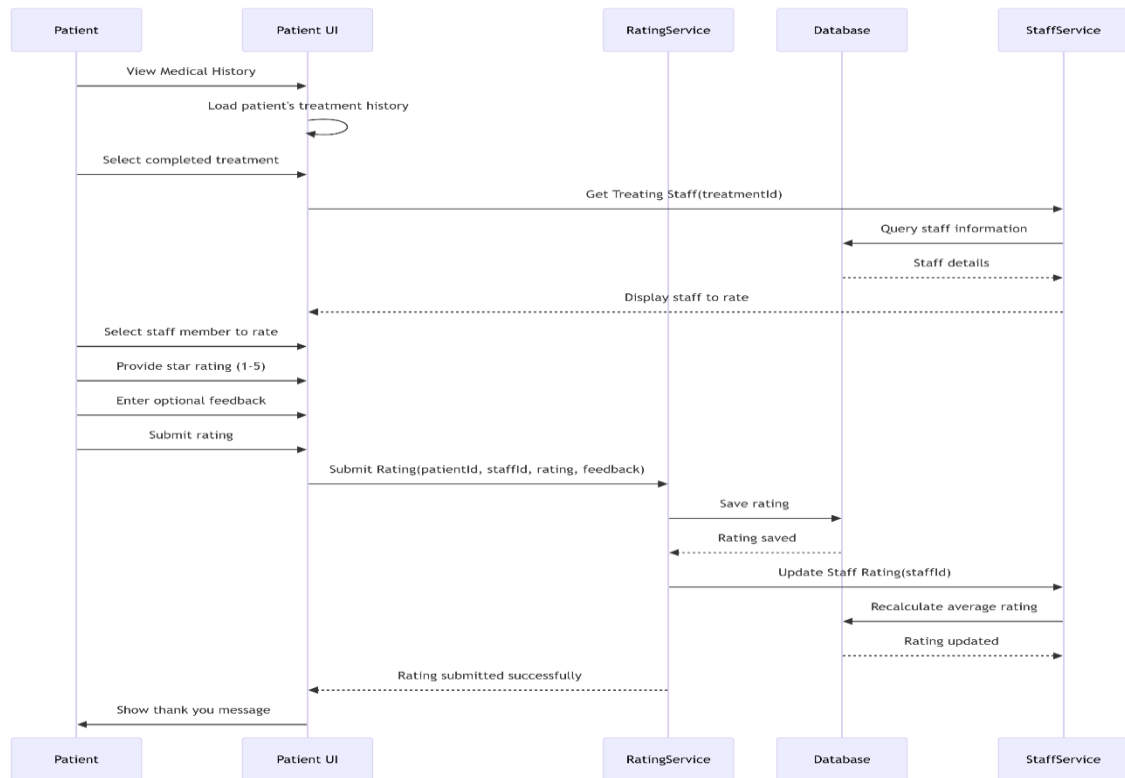


Figure 21: Patient Rating System

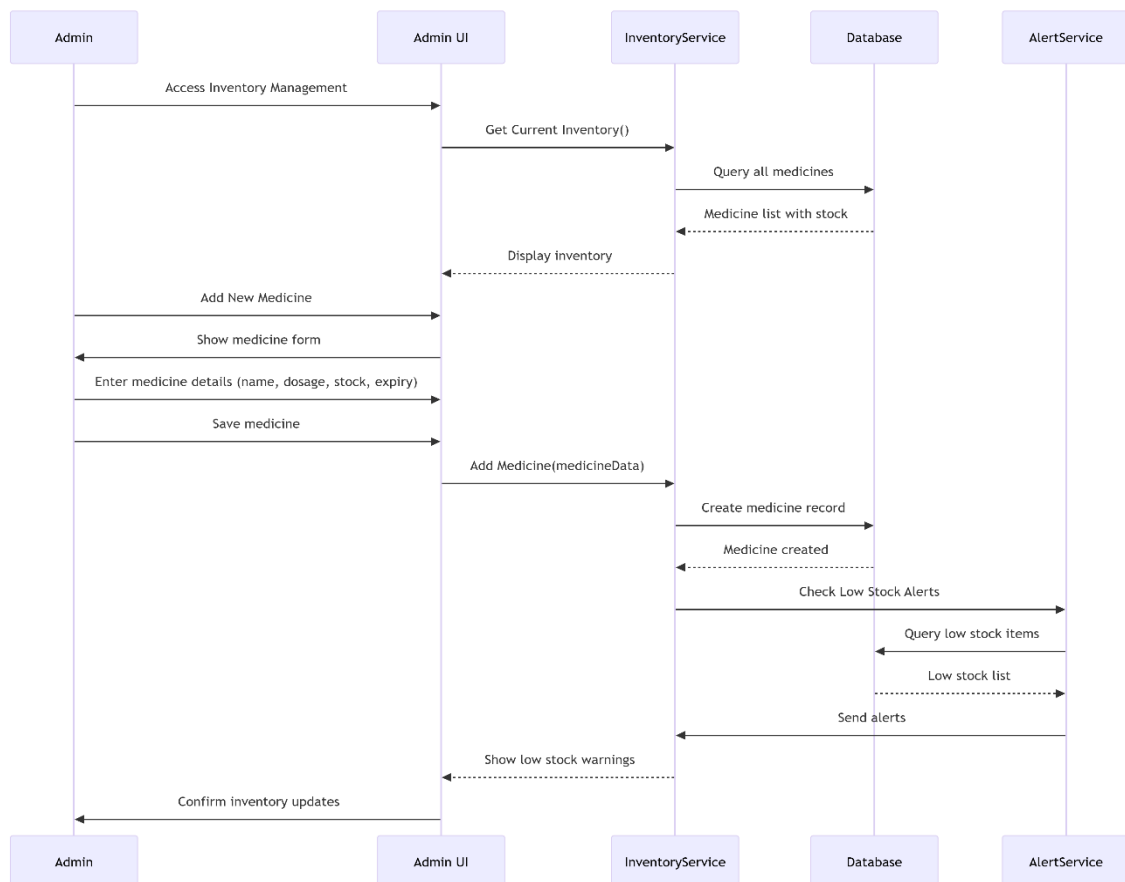


Figure 22: Admin Managing Inventory

10.6 Database

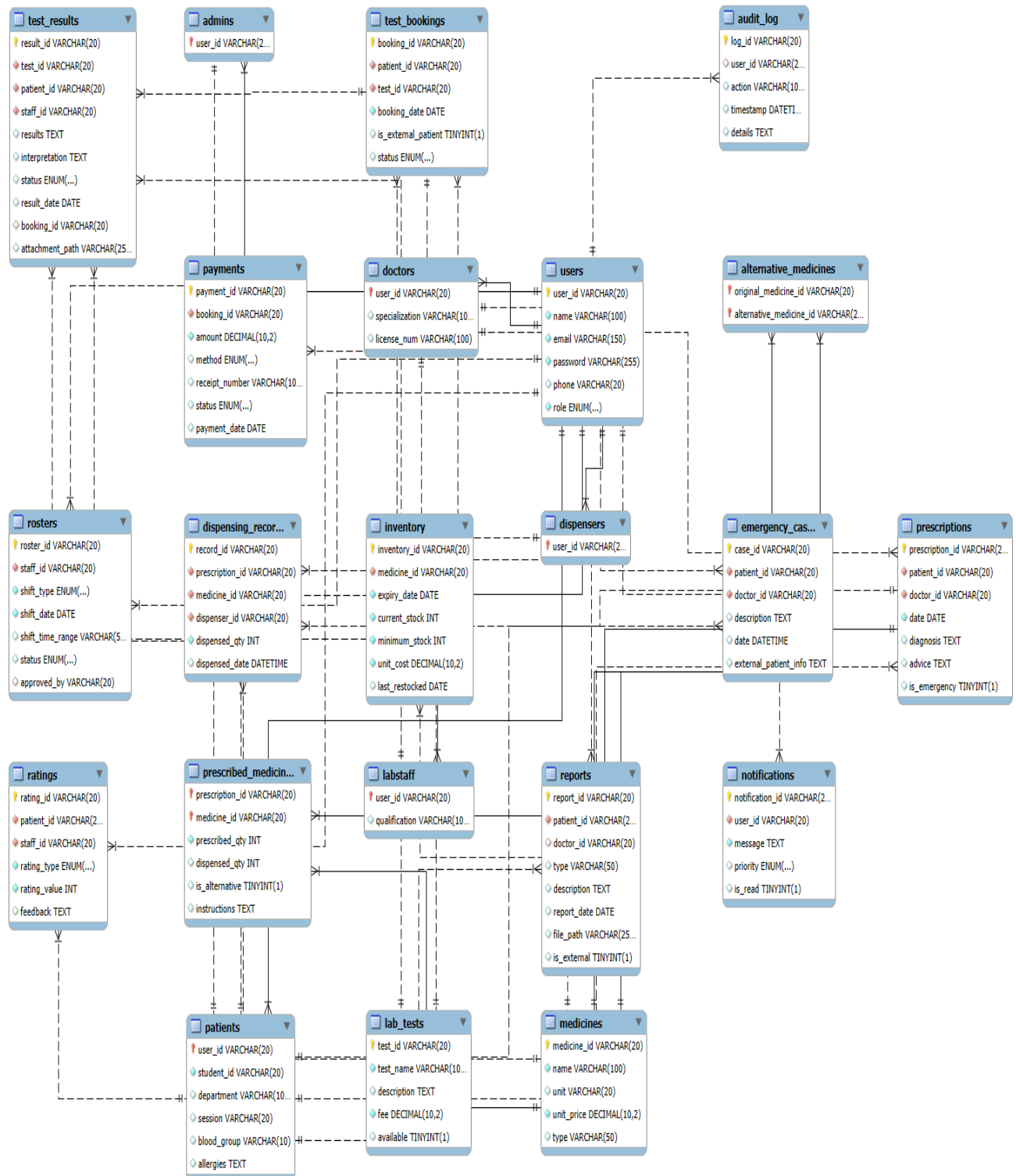


Figure 23: Database Schema Diagram